Path-centric Cardinality Estimation for Subgraph Matching

Zhengdong Wang
Shanghai Jiao Tong University
lnwzd2009@sjtu.edu.cn

Qiang Yin
Shanghai Jiao Tong University
q.yin@sjtu.edu.cn

Longbin Lai
Alibaba Group
longbin.lailb@alibaba-inc.com

ABSTRACT

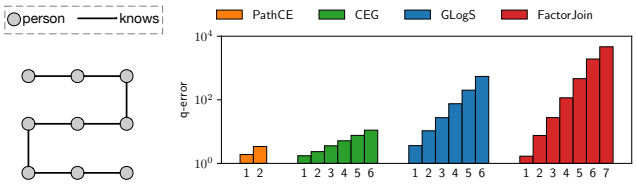
This paper presents PathCE, a path-centric cardinality estimation framework for subgraph matching. PathCE improves estimation accuracy by utilizing statistics from short graph queries. At its core is a novel data structure called the *path-centric summary graph* (PSG), which captures short path query statistics from a data graph G and represents them in a new graph $\mathcal{G}$. Given a graph query Q and a PSG graph $\mathcal{G}$ for G, PathCE decomposes Q into a simpler query $\mathcal{Q}$, where each edge in $\mathcal{Q}$ corresponds to a sub-path query in Q with statistics included in $\mathcal{G}$. PathCE estimates the cardinality using $\mathcal{Q}$ and $\mathcal{G}$, requiring significantly fewer estimation iterations while ensuring that the estimate remains an upper bound on the true cardinality of $Q(G)$. It also includes PSGBuilder, a parallelly scalable algorithm that constructs PSG's for any given graph in linear time, efficiently scaling with the number of processors. Empirical results on real-world and synthetic datasets show that PathCE outperforms state-of-the-art baselines in accuracy, estimation latency, and summary construction efficiency.

1 INTRODUCTION

Given a graph query Q and a data graph G, *subgraph matching* is to find all matches of Q in G. It serves as a fundamental component for modern graph query languages such as SPARQL [16], Cypher [13], and the most recently released ISO/IEC standard GQL [11]. Cardinality estimation for subgraph matching is to estimate the number of matches of Q in G, denoted as $|Q(G)|$, without explicitly computing them. It plays a pivotal role in cost-based query optimizers, which rely on it to estimate the cost of candidate query plans [24].

As highlighted in recent studies, while cardinality estimation has been extensively explored in relational databases, research on graph-specific estimation techniques remains relatively underdeveloped [5, 9, 33]. Although graph queries can be expressed as joins in relational frameworks, directly applying relational methods to graph data introduces significant challenges. Relational DBMSs typically focus on table-level statistics [48], which are analogous to edge-level statistics in graph terms. However, collecting detailed statistics such as join counts or maximal degrees across multiple tables is often impractical, impeding accurate cardinality estimation for complex graph queries [23]. In contrast, graph DBMSs efficiently support neighborhood access, enabling the collection of statistics on small graph patterns, such as paths [33]. This opens up new opportunities for designing graph-based cardinality estimators.

The summary-based cardinality estimators, which compute statistics for cardinality estimation prior to query execution, are often favored in practice because it relies solely on the data rather than specific queries [5, 15, 33]. In the context of subgraph matching, summary-based cardinality estimators use pre-computed statistics of small queries on G to estimate $|Q(G)|$ and usually employ an iterative estimation approach. Specifically, each iteration generates estimates for a subquery of Q, referred to as an *estimation iteration*. However, we find that existing summary-based estimators, both graph-based and relational-based, such as CEG [5], GLogS [23], and FactorJoin [48] face significant accuracy challenges especially when handling complex graph queries, which are common in real-life workloads [8, 29], as illustrated below.



(a) Query Q **(b)** Estimation accuracy of subqueries across iterations

Figure 1: Cardinality estimation using graph query Q on LDBC. For each estimator, (b) shows the q-error incurred during each iteration.

Example 1: Consider cardinality estimation using a graph query Q on the LDBC dataset [8], which is developed by the Linked Data Benchmark Council to benchmark graph DBMSs. As shown in Fig. 1(a), Q has 9 vertices labeled person, and 8 edges labeled knows.

(1) The number of estimation iterations varies among estimators. For example, FactorJoin generates estimates for the subquery Q_i in the i-th iteration, where Q_i is a subquery of Q consisting of i+2 vertices labeled person and i+1 edges labeled knows. Thus, FactorJoin takes 7 iterations to produce the estimate of Q, while PathCE, CEG and GLogS require 2, 6 and 6 iterations, respectively.

(2) For each estimator, we break down its estimation process and analyze the subquery estimates across different iterations. We find that the number of iterations strongly affects estimation accuracy. We employ the classic accuracy metric q-error [30] to measure deviations of estimated cardinality from true cardinality. Figure 1(b) shows how q-error evolves across iterations for each estimator. □

Observation and challenges. We observe that q-error increases progressively in later iterations due to *error accumulation*, as later subquery estimates rely on earlier (potentially inaccurate) results. This effect is particularly evident in estimators requiring many iterations. For example, in FactorJoin, the q-error grows from 1.69 to 4657.23 across its 7 iterations for the query in Fig. 1(a). In contrast, PathCE needs 2 iterations, with q-error rising from 1.90 to 3.39. These findings suggest that *minimizing the number of estimation iterations is crucial for reducing cumulative estimation error and improving accuracy*. However, this also raises several challenges.

- (**C1**) Is there an effective way to reduce the number of estimation iterations and improve accuracy? Prior research indicates that utilizing statistics from small graph queries, *e.g.*, 2-path queries and triangle queries, can improve accuracy [5, 23].
- (**C2**) Is there a scalable way to construct sufficient statistics while substantially reducing estimation iterations? It is clear that the estimation accuracy can be improved using statistics of more generic queries, but concerns about the efficiency of computing these statistics remain [23, 33, 38]. For example, al-

though GLogS [23] can maintain any query statistics theoretically, the cost is prohibitive. Thus, GLogS still relies on small queries, requiring extensive estimation iterations for processing the aforementioned graph query.

- **(C3)** Another critical argument from previous studies [3, 10, 48] highlights the value of upper bound estimates in query optimization, which has also been demonstrated in our experiments (Sec. 8), where FactorJoin, despite having lower estimation accuracy, often surpasses CEG and GLogS in generating more effective execution plans. This leads us to consider: Can we ensure estimators constantly provide an upper bound on the true cardinality?

In this paper, we propose PathCE, a *path-centric* summary-based cardinality estimation framework, to answer these questions.

A path-centric approach. Given a graph query Q, PathCE first decomposes Q into a collection of path queries, forming a new query $\mathcal{Q}$, where each edge of $\mathcal{Q}$ represents a sub-path in Q. Estimation is conducted using $\mathcal{Q}$ and the path query statistics, which addresses **(C1)** by significantly reducing the number of iterations due to $\mathcal{Q}$'s simpler structure, thereby enhancing estimation accuracy. To tackle **(C2)**, PathCE introduces a scalable algorithm to efficiently construct path query statistics. This naturally balances accuracy and efficiency: offering higher accuracy compared to edge-only statistics [48], while being more efficient than using statistics of generic graph queries [23]. Finally, inspired by recent advancements in upper bound estimation [1, 3], PathCE addresses **(C3)** by integrating both count and maximal-degree information into the path query statistics to ensure an upper bound estimate.

Below, we summarize our contributions and paper organization.

Contribution & organization. We develop a path-centric cardinality estimation framework PathCE to improve estimation accuracy, while maintaining the efficiency of computing the statistics.

(1) A novel summary structure (Sec. 3). We introduce *path-centric summary graphs* (PSG) as a novel data structure for maintaining the statistics of path queries. A PSG for a graph G is itself a graph $\mathcal{G}$, where (i) each vertex in $\mathcal{G}$ represents a subset of vertices in G with an identical label, and (ii) each edge in $\mathcal{G}$ represents a path query between the vertex subsets at its endpoints. The size of $\mathcal{G}$ depends only on types of vertices/edges in G rather than the size of G, making PSG a scalable summary structure for large graphs.

(2) A path-centric framework (Sec. 4). Given a query Q and a summary graph $\mathcal{G}$, PathCE estimates $|Q(G)|$ in two steps. PathCE first generates an estimation scheme $(\mathcal{Q}, \mathcal{S})$ for Q, consisting of a new query $\mathcal{Q}$ and an estimation order $\mathcal{S}$. The query $\mathcal{Q}$ has a simpler structure than Q, and $\mathcal{S}$ is a vertex sequence of $\mathcal{Q}$. PathCE next conducts estimation using $\mathcal{Q}$ and $\mathcal{G}$. This significantly reduces estimation iterations since $\mathcal{Q}$ is much simpler. In addition, the estimation is guided by $\mathcal{S}$, avoiding the exhaustive search for better estimates.

(3) Cardinality estimation with PSG (Sec. 5 & Sec. 6). We develop a systematic approach to generate an effective estimation scheme $(\mathcal{Q}, \mathcal{S})$ for a given query Q (Sec. 5). To achieve upper bound estimation, we introduce a *graph-native* approach that guarantees every estimation iteration produces an upper bound estimate (Sec. 6).

(4) Scalable PSG *construction* (Sec. 7). We develop a *parallelly scalable* [22] algorithm PSGBuilder to construct PSG's. The algorithm PSGBuilder runs in linear time *w.r.t.* the size of G and guarantees to reduce the running time when working with more processors.

(5) Experimental study (Sec. 8). We prototype PathCE and compare it with state-of-the-art summary-based, sampling-based and ML-based baselines. We find the following. **(a)** Among the summary-based estimators, PathCE produces the most accurate estimates for cyclic queries on both real-world and synthetic datasets. For acyclic queries, PathCE achieves comparable accuracy to CEG, *e.g.,*PathCE achieves an average q-error below 20 on IMDB. The performance of PathCE is quite stable: it always produces upper bound estimates, its q-error grows slowly with query size, and remains consistent *w.r.t.* query topology (Exp-1). **(b)** PathCE is one of the fastest estimators in terms of estimation latency. It completes all test queries within 0.1 seconds, with low variance (Exp-2). **(c)** The PSGBuilder algorithm of PathCE is highly efficient. It constructs a PSG graph for the LDBC dataset in 734 seconds and scales well with both graph size and number of thread (Exp-3 & Exp-4). **(d)** PathCE consistently beats all baselines in end-to-end performance, with lower latencies in both query planning and query execution (Exp-5).

Related work. We categorize the related work as follows.

Summary-based estimators. Summary-based estimators have been widely adopted by both relational and graph databases.

(1) Summary-based estimators for *relational databases* utilize summaries, also referred to as *catalogs* or *sketches* in some literatures, such as histograms [20], HyperLogLog [12], and CountMin [7], in combination with statistical assumptions on underlying data to make estimates, following the paradigm pioneered by the System R optimizer [36]. Closer to this work are BoundSketch [3], Simplicity [18], FactorJoin [48], and SafeBound [10], which utilize maximal-degree statistics to produce upper bound estimates.

PathCE differs from prior work in the following. (a) Unlike the System R style estimator that relies on statistics assumptions, PathCE relies on no statistics assumption and is essentially derived from the degree-constraint bound [1], which guarantees upper bound estimates. using maximal-degree statistics. (b) PathCE collects statistics of short path queries, in contrast, most prior work relies on single edge (or base table), *e.g.,* [3, 10, 18, 48]. (c) PathCE produces upper bound estimates by a graph-base approach, as opposed to the bound formula generator of BoundSketch [3], or probabilistic graphical models adopted by FactorJoin [48].

(2) Summary-based estimators have also been developed for *graph databases* [5, 9, 23, 32, 38, 47]. C-SET [32] introduces the *characteristic set*, which represents the distinct outgoing labels for each vertex in an RDF graph. SumRDF [38] proposes a *typed summary* graph that groups vertices with similar labels and incident edge distributions. GLogS [23] presents a graph-structured summary retaining small subgraph counts, while CEG [5] uses a *Markov table* to store short path query cardinalities. A-LHD [47] collects single-edge statistics and adopts a label probability propagation model to improve accuracy. Color [9]is a recent summary-based estimator that exploits graph coloring techniques in cardinality estimation[1]. Like System R style estimators, they

[1]Here we refer to the default variant of Color that uses average degrees.

Table 1: Notations

$G = (V, E, L)$, Π_V, $\mathbb{P}$	graph, vertex partition, path query set
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{L})$, $\mathcal{Q}$	PSG and PSG query
$Q = (V_Q, E_Q, L_Q)$, $Q(G)$	graph query and its match set on G
$Q[u_1, u_2]$, $Q[u]$	graph query with designated endpoints

face similar challenges due to reliance on assumptions.

PathCE differs from prior work in several ways. (a) PathCE is assumption-free and guarantees upper bound estimates by leveraging maximal-degree statistics. (b) PathCE uses short path queries, unlike C-SET, SumRDF and Color, which only consider single edge queries. (c) While both CEG and GLogS collect statistics on 2-path queries, they do not consider maximal-degree information. (d) CEG and GLogS also utilize triangle counts to improve accuracy, but collecting these counts is cost-prohibitive, requiring techniques like graph sparsification [34, 41]. In contrast, PathCE constructs summaries efficiently without sacrificing estimation accuracy. (d) While PathCE employs the heuristic GBSA [48] for vertex partitioning, Color utilizes graph coloring to achieve the same goal.

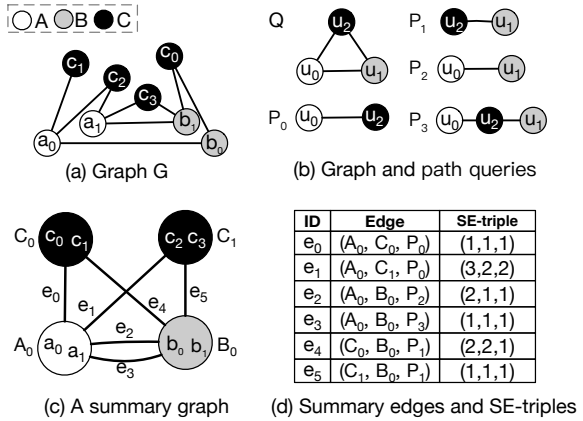
Sampling-based and ML-based estimators. Sampling-based estimators are alternatives to summary-based estimators, and they have played an important role in query optimization for decades [26, 43, 52]. Generally, they are able to handle queries with correlations and non-uniformity better than summary-based estimators, but with problems of high estimation latency and poor scalability in terms of the query size [10]. Recently, ML-based *query-driven* [17, 21, 31, 51] and *data-driven* [35, 44, 49, 53] estimators have been actively developed and they have achieved satisfactory accuracy in open benchmarks [14, 40]. Latest work on ML-based estimators propose pretrained models and fine-tuning [6, 27, 50] to avoid costly model retraining over changing data and workloads. As a summary-based estimator, PathCE generally offers (a) better estimation latency compared to sampling-based one, (b) scalable summary construction without extensive model training, and (c) interpretable results, unlike the black-box nature of ML-based estimators [45].

2 PRELIMINARIES

We begin with basic notations. Let Γ be a countable set of labels.

Graphs and queries. We follow the typical setting of subgraph matching [33] and focus on *undirected* labeled graphs. Our approach can be easily extended to directed graphs.

- A *graph* $G = (V, E, L)$ is a triple, where V is a finite vertex set, $E \subseteq V \times V \times \Gamma$ is a finite set of labeled edges, and L is a *labeling function* that maps each vertex $v \in V$ (and each edge $e \in E$) to a label from Γ. For an edge $e = (v_1, v_2, \ell)$, we have $\ell = L(e)$, with v_1 and v_2 referred to as the endpoints of e. For a vertex v, let $\mathsf{Nbr}(v)$ denote the neighbors of v in G.
- A graph query Q is often represented by its query graph $Q = (V_Q, E_Q, L_Q)$, where V_Q, E_Q and L_Q are its vertex set, edge set and labeling function. A query Q' is a *subquery* of Q if its query graph is a subgraph of Q's query graph. Q is *cyclic* if its query graph contains a cycle as a subgraph; otherwise Q is *cyclic*.
- A *path query* is a special graph query whose query graph forms a labeled path. We use the symbol P for path queries. A path query with k edges is called a *k-path query* and can be represented as $u_0e_1u_1 \ldots u_{k-1}e_ku_k$, where each edge e_i connects endpoints u_{i-1} and u_i for $i = 1, \ldots, k$. We refer to u_0 and u_k as the *head* and *tail* of P, denoted by h_P and t_P, respectively.



ID	Edge	SE-triple
e_0	(A_0, C_0, P_0)	(1,1,1)
e_1	(A_0, C_1, P_0)	(3,2,2)
e_2	(A_0, B_0, P_2)	(2,1,1)
e_3	(A_0, B_0, P_3)	(1,1,1)
e_4	(C_0, B_0, P_1)	(2,2,1)
e_5	(C_1, B_0, P_1)	(1,1,1)

Figure 2: A data graph G, example graph and path queries, along with a path-centric summary graph $\mathcal{G}$ of G.

Subgraph matching. Given a graph query $Q = (V_Q, E_Q, L_Q)$ and a data graph $G = (V, E, L)$, a *subgraph homomorphic match* of Q in G is a mapping $\rho : V_Q \to V$ such that (i) for every $u \in V_Q$, $L_Q(u) = L(\rho(u))$; and (ii) for every edge (u_1, u_2, ℓ) in E_Q, there exists an edge $(\rho(u_1), \rho(u_2), \ell')$ in E with $\ell = \ell'$. For simplicity, we will refer to ρ as a *match* of Q in G. Note that there may be multiple matches of Q in G. We denote by $Q(G)$ the set of all matches of Q in G. Similarly, $P(G)$ is defined for a path query P. Given a query graph Q and a data graph G, the *subgraph matching problem* is to compute $Q(G)$.

Cardinality estimation problem. Given a query graph Q and a data graph G, the cardinality of $Q(G)$, denoted by $|Q(G)|$, refers to the number of matches of Q in G. The *cardinality estimation problem for subgraph matching* is to estimate $|Q(G)|$ without explicitly computing the set $Q(G)$. Such estimation is typically performed by leveraging statistics from smaller subqueries [5, 23, 48]. As we will see shortly, we collect statistics of short path queries only and utilize them to support cardinality estimation for arbitrary graph queries.

Example 2: Consider a data graph G as shown in Fig. 2(a), with three vertex labels A, B, and C, represented by different colors. The edge labels are assumed to be identical and thus are omitted.

(1) We have $|Q(G)| = 1$ for the graph query Q shown in Fig. 2(b), as $Q(G)$ contains exactly one match ρ_0, *i.e.,* $Q(G) = \{\rho_0\}$, where $\rho_0 = \{(u_0, a_1), (u_1, b_1), (u_2, c_3)\}$.

(2) For the path query P_1 shown in Fig. 2(b), we have $|P_1(G)| = 3$, with the following three matches: $\rho_1 = \{(u_2, c_0), (u_1, b_0)\}$, $\rho_2 = \{(u_2, c_0), (u_1, b_1)\}$, and $\rho_3 = \{(u_2, c_3), (u_1, b_1)\}$. Furthermore, the head and tail vertices of the path query P_1 are u_2 and u_1, respectively; that is, $h_{P_1} = u_2$ and $t_{P_1} = u_1$. □

The notations of this paper are summarized in Table 1.

3 PATH-CENTRIC SUMMARY GRAPHS

In this section, we introduce *path-centric summary graphs* (PSG), a data structure that maintains statistics of path queries as a graph.

We first introduce a general framework for defining the statistics of a graph query Q. Note that these statistics are always defined with respect to an underlying data graph. To simplify our presentation, we assume a fixed data graph $G = (V, E, L)$ throughout this section.

Definition 3.1: Let Q be a graph query, and let u_1 and u_2 be two designated nodes in Q. Let V_1 and V_2 be two subsets of V. The

summary of $Q[u_1, u_2]$ with respect to the vertex subset pair (V_1, V_2) is defined as a triple (c, d_1, d_2), where:

- $c = \big|\{\rho \in Q(G) \mid \rho(u_1) \in V_1,\ \rho(u_2) \in V_2\}\big|$;
- $d_1 = \max_{v_1 \in V_1} \big|\{\rho \in Q(G) \mid \rho(u_1) = v_1,\ \rho(u_2) \in V_2\}\big|$;
- $d_2 = \max_{v_2 \in V_2} \big|\{\rho \in Q(G) \mid \rho(u_1) \in V_1,\ \rho(u_2) = v_2\}\big|$.

Intuitively, c denotes the number of matches in $Q(G)$ where u_1 and u_2 are mapped to vertices in V_1 and V_2, respectively. In these matches, d_1 (resp. d_2) represents the maximum number of occurrences of any vertex in V_1 (resp. V_2) as the match for u_1 (resp. u_2). We refer to d_1 and d_2 as the *maximal-degree statistics* of $Q[u_1, u_2]$. □

Similarly, we define the summary for a single designated node.

Definition 3.2: Let Q be a graph query, u_1 be a designated node in Q, and $V_1 \subseteq V$ be a vertex subset. The summary of $Q[u_1]$ with respect to V_1 is also a triple (c, d_1, d_2), where:

- $c = \big|\{\rho \in Q(G) \mid \rho(u_1) \in V_1\}\big|$;
- $d_1 = \max_{v_1 \in V_1} \big|\{\rho \in Q(G) \mid \rho(u_1) = v_1\}\big|$;
- $d_2 = \infty$, *i.e.,* d_2 is undefined as Q has only one designated node. □

Remark. Intuitively, we treat $Q[u_1, u_2]$ as an *edge-like* query by designating u_1 and u_2 as its two endpoints. Similarly, $Q[u_1]$ can be viewed as a *vertex-like* query, with u_1 as its sole endpoint. □

Example 3: Continue from Example 2 and consider the path query P_1 depicted in Fig. 2(b). Let $B_0 = \{b_0, b_1\}$ and $C_0 = \{c_0, c_1\}$ be two subsets of V. We can observe the following.

(1) The summary of $P_1[u_2, u_1]$ *w.r.t.* (C_0, B_0) is $(c, d_1, d_2) = (2, 2, 1)$. Indeed, among the three matches in $P_1(G)$, only ρ_1 and ρ_2 satisfy the additional constraint that u_2 is matched to a vertex in C_0 and u_1 is matched to a vertex in B_0 (see the definitions of ρ_1 and ρ_2 in Example 2); $d_1 = 2$ since both ρ_1 and ρ_2 map u_2 to the vertex c_0, which has the maximal number of occurrences; similarly, $d_2 = 1$ since both b_0 and b_1 occur once as the match of u_1 in these matches.

(2) The summary of $P_1[u_1]$ *w.r.t.* B_0 is $(3, 2, \infty)$. This is because all three matches of $P_1(G)$, *i.e.,* ρ_1, ρ_2, and ρ_3, map u_1 to a vertex in B_0, and the vertex b_1 in B_0 appears twice, which has the maximal number of occurrences among these matches. □

To construct a PSG for a data graph G, we utilize a *summary vertex partition* Π_V to partition V into a collection of disjoint subsets. The vertices in each subset have an identical label and each subset is referred to as a *summary vertex*. Specifically, let V^ℓ be the set of vertices in V with label ℓ. Given a predefined number M, Π_V divides each V^ℓ into M summary vertices $V_1^\ell, \ldots, V_M^\ell$. Thus, Π_V can be represented as $\Pi_V = \{V_i^\ell \mid \ell \text{ is a vertex label in } G,\ i = 1, \ldots, M\}$.

Collecting statistics for all possible graph queries, even just for path queries, is computationally infeasible [5, 23]. Thus we focus on a small set $\mathbb{P}$ of k-path queries (with small k), representing their statistics with a graph-based data structure called the *path-centric summary graph* (PSG). These path query statistics will then be used to support cardinality estimation for arbitrary graph queries.

Definition 3.3: Given a summary vertex partition Π_V of G, and a set $\mathbb{P}$ of path queries, a *path-centric summary graph* (PSG) of G *w.r.t.* Π_V and $\mathbb{P}$ is a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{L})$ satisfying the followings. (1) The *vertex set* $\mathcal{V}$ consists of all the summary vertices defined by Π_V. Observe that all vertices in a summary vertex V' share an identical label ℓ. Thus, we let $\mathcal{L}(V') = \ell$.

(2). The *edge set* $\mathcal{E}$ consists of all edges $e = (V_1, V_2, P)$ such that (i) P is a path query from the set $\mathbb{P}$, which serves as the label of e, and (ii) the labels $\mathcal{L}(V_1)$ and $\mathcal{L}(V_2)$ match the labels of h_P and t_P, respectively. Each edge in $\mathcal{E}$ is referred to as a *summary edge*.

(3) Each summary edge $e = (V_1, V_2, P)$ carries a triple (c, d_1, d_2) as its property, which is precisely the summary of the path query $P[h_P, t_P]$ with respect to (V_1, V_2), where the head vertex h_P and tail vertex t_P are taken as the designated nodes of P.

We refer to the property triple associated with a summary edge as an SE-*triple*. The collection of all SE-triples associated with summary edges of the form (V_1, V_2, P) in $\mathcal{G}$ is called the *statistics of* P. □

Intuitively, a PSG adopts a path-centric approach, representing the statistics of *path queries* within a graph structure. The size of a PSG is independent of the size of the data graph G and is bounded by $O(NM^2)$, where N is the number of path queries in $\mathbb{P}$, and M is the number of summary vertices per label. This property makes PSG a scalable summary data structure for large graphs.

Example 4: Continue from Example 2 and Example 3, and let $\Pi_V = \{A_0, B_0, C_0, C_1\}$ be a vertex partition with 4 summary vertices, where $A_0 = \{a_0, a_1\}$, $B_0 = \{b_0, b_1\}$, $C_0 = \{c_0, c_1\}$, and $C_1 = \{c_2, c_3\}$ (see Fig. 2(c)). Let $\mathbb{P} = \{P_0, P_1, P_2, P_3\}$ denote the set of all path queries shown in Fig. 2(b). The summary graph $\mathcal{G}$ with respect to Π_V and $\mathbb{P}$ is depicted in Fig. 2(c), and Fig 2(d) lists its 5 summary edges, along with their labels and SE-triples. Observe the following.

(1) The SE-triple of the summary edge e_4 is $(2, 2, 1)$, as it corresponds to the summary of $P_1[u_2, u_1]$ with respect to the summary vertex pair (C_0, B_0). Similarly, one can verify the other SE-triples.

(2) There are two path queries in $\mathbb{P}$, namely P_2 and P_3, that connect a vertex labeled A to a vertex labeled B. Consequently, two summary edges, e_2 and e_3, exist in $\mathcal{G}$ between the summary vertices A_0 and B_0. The SE-triple associated with e_2 represents the summary of $P_2[u_0, u_1]$ with respect to (A_0, B_0), while the SE-triple of e_3 represents the summary of $P_3[u_0, u_1]$ (see more in Fig. 2(d)). □

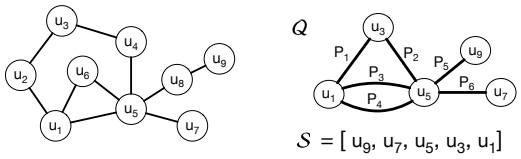
We often need to represent the statistics of longer path queries beyond $\mathbb{P}$, as well as general graph queries. Fortunately, our framework allows all such queries to be represented in PSG.

(1) For $Q[u_1, u_2]$ with u_1 and u_2 as its designated endpoints, we add a new summary edge $(V_1, V_2, Q[u_1, u_2])$ in $\mathcal{G}$, for every summary vertex pair (V_1, V_2) with $\mathcal{L}(V_1) = L(u_1)$ and $\mathcal{L}(V_2) = L(u_2)$. Its SE-triple is defined as the summary of $Q[u_1, u_2]$ *w.r.t.* (V_1, V_2).

(2) For $Q[u_1]$ with u_1 as its sole designated endpoint, we first introduce a *dummy summary vertex* U to the summary vertex set $\mathcal{V}$, if it does not already exist. Then, for each summary vertex V_1 with $\mathcal{L}(V_1) = L(u_1)$, we add a new summary edge $(V_1, U, Q[u_1])$, whose SE-triple is the summary of $Q[u_1]$ with respect to V_1.

Similarly, the set of SE-triples associated with these edges constitutes the statistics of $Q[u_1, u_2]$ and $Q[u_1]$, respectively.

Cardinality estimation with PSG's. Given a query Q and a path-centric summary graph $\mathcal{G}$ of G, our goal is to estimate $|Q(G)|$ by making use of the path query statistics included in $\mathcal{G}$. Recall we only include the statistics of a small set of k-path queries when



(a) A query Q (b) An estimation scheme of Q

Figure 3: A graph query Q and its estimation scheme $(\mathcal{Q}, \mathcal{S})$.

Algorithm 1: PathCE: Path-centric estimation workflow

Input: A graph query Q and a PSG graph $\mathcal{G}$ of data graph G.
Output: An upper bound estimaiton of $|Q(G)|$.

1 $(\mathcal{Q}, \mathcal{S}) \leftarrow$ GenScheme$(Q, \mathcal{G})$; // $\mathcal{S} = [u_1, \ldots, u_k]$
2 **for** $i = 1, \ldots, k-1$ **do**
3 $\quad Q_i[\mathsf{Nbr}(u_i)] \leftarrow$ ExtraSubQ(Q, u_i); //*Extract a subquery*
4 $\quad \mathcal{G} \leftarrow \mathcal{G} \cup$ EstTriple$(\mathcal{G}, Q_i[\mathsf{Nbr}(u_i)])$; //*Estimate* SE-*triple*
5 $\quad \mathcal{Q} \leftarrow$ ReduceQuery$(\mathcal{Q}, Q_i[\mathsf{Nbr}(u_i)])$; //*Reduce* PSG *query*
6 **return** AggCnt$(\mathcal{G}, Q, u_k)$;

constructing $\mathcal{G}$. If $\mathcal{G}$ already contains statistics for Q, *e.g.,* if Q is a small path query in $\mathbb{P}$, we aggregate the statistics to estimate $|Q(G)|$. For longer paths or generic graph queries like Q, we estimate their statistics using the shorter path statistics contained in $\mathcal{G}$. Once we have the statistics for Q, we aggregate them to estimate $|Q(G)|$.

Remark. Our framework restricts the number of designated endpoints in a graph query Q to at most two (see Definitions 3.1, 3.2, and 3.3). While this design can be extended to support more endpoints, we make this choice deliberately for the following reasons.

(1) Limiting to two endpoints allows all summaries to be represented using a simple graph structure. Supporting more endpoints would require hypergraphs, significantly increasing design complexity.

(2) This restriction also reduces both storage and computation costs. A path query with two endpoints requires $O(M^2)$ space in PSG. Allowing three endpoints would increase this to $O(M^3)$. Moreover, more designated endpoints requires additional maximal-degree statistics, further increasing construction overhead (see Sec. 7). □

4 PATH-CENTRIC ESTIMATION FRAMEWORK

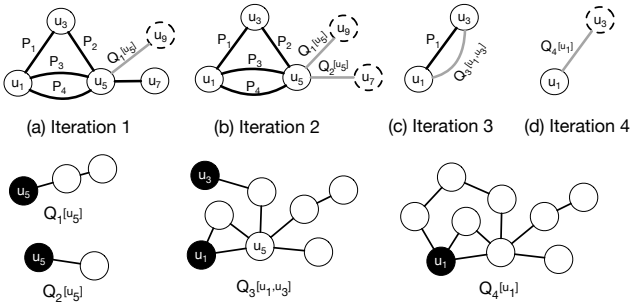
We present PathCE in this section, a path-centric cardinality estimation framework. We begin with an overview of PathCE (Sec.4.1), followed by the key challenges in developing PathCE (Sec.4.2).

4.1 Cardinality Estimation Workflow

Taking a graph query Q and a summary graph $\mathcal{G}$ of G as input, PathCE outputs an estimate of $|Q(G)|$. Algorithm 1 outlines the overall workflow of PathCE, which consists of an *estimation scheme generation* step (line 1) and a *cardinality estimation* step (lines 2–6).

Estimation scheme generation. PathCE decomposes the input query Q and generates an estimation scheme for it. This scheme is represented as a pair $(\mathcal{Q}, \mathcal{S})$ (see Fig. 3(b) for an example), where (i) $\mathcal{Q}$ is also a query such that every edge of $\mathcal{Q}$ represents a sub-path query of Q whose statistics have been included in $\mathcal{G}$, and (ii) $\mathcal{S}$ specifies an *estimation order* $[u_1, \ldots, u_k]$ that will be used in the estimation step (see below), where $u_1, \ldots, u_k$ are the vertices of $\mathcal{Q}$. We will refer to $\mathcal{Q}$ as a PSG *query of* Q, to highlight that PathCE utilizes $\mathcal{Q}$ directly over the PSG graph $\mathcal{G}$ to estimate $|Q(G)|$.

Example 5: Suppose that $\mathcal{G}$ includes statistics of all k-path queries



(e) Subqueries processed in iterations; designated endpoints are marked in black.

Figure 4: Iterative cardinality estimation. A dotted circle indicates a dummy vertex, while a gray edge represents a subquery.

for $k \leq 2$. A graph query Q and its estimation scheme $(\mathcal{Q}, \mathcal{S})$ are shown in Figure 3. Every edge in $\mathcal{Q}$ represents a path of Q, *e.g.,* the edges labeled with P_1 and P_2 represent the two paths u_1–u_2–u_3 and u_3–u_4–u_5 of Q, respectively. The estimation order is $\mathcal{S} = [u_9, u_7, u_5, u_3, u_1]$. Note that $\mathcal{Q}$ has a simpler structure than Q, *e.g.,* $\mathcal{Q}$ has 5 vertices and 6 edge, whereas Q has 9 vertices and 10 edges. □

As we will show shortly, PathCE constructs an estimation scheme $(\mathcal{Q}, \mathcal{S})$ for a given query Q by explicitly optimizing the number of estimation iterations. This brings two-fold benefits. First, by replacing long paths in Q with edges in $\mathcal{Q}$, PathCE can directly leverage the path query statistics encoded in $\mathcal{G}$. This transformation reduces the number of estimation iterations, significantly *lowering cumulative estimation error and improving accuracy*. Second, the estimation process is guided by the estimation order $\mathcal{S}$, which avoids exhaustive search and also requires fewer iterations. This greatly *reduces estimation latency*.

Iterative cardinality estimation. Guided by $(\mathcal{Q}, \mathcal{S})$, PathCE next estimates $|Q(G)|$ by making use of the path query statistics included in $\mathcal{G}$. PathCE performs the estimation in an iterative manner, as also adopted in prior studies [18, 48]. One key difference is that, PathCE uses $\mathcal{Q}$ to leverage path query statistics included in $\mathcal{G}$ and reduce the number of estimation iterations, while prior approaches usually estimate directly with Q and require more iterations.

Let $\mathcal{S} = [u_1, \ldots, u_k]$ be the estimation order. As outlined in Algorithm 1, in the i-th iteration PathCE estimates the statistics of a subquery Q_i of $\mathcal{Q}$ (lines 2-5). The subquery Q_i takes $\mathsf{Nbr}(u_i)$ as its designated endpoints, where $\mathsf{Nbr}(u_i)$ are u_i's neighbors in Q_i. In particular, each iteration involves the following three steps.

(1) Extract a subquery. Let $\mathsf{Nbr}(u_i)$ be the neighbors of u_i in the PSG query $\mathcal{Q}$. PathCE first extracts a subquery Q_i from Q, using $\mathsf{Nbr}(u_i)$ as the designated endpoints (line 3). The subquery $Q_i[\mathsf{Nbr}(u_i)]$ consists of the edges covered by u_i's incident edges in $\mathcal{Q}$. For example, in the PSG query $\mathcal{Q}$ shown in Fig. 3(b), PathCE processes u_9 in the first iteration. Note that $\mathsf{Nbr}(u_9) = \{u_5\}$, and u_9 has only one incident edge labeled P_5; thus, as shown in Fig. 6(e), the subquery $Q_1[u_5]$ consists of two edges covered by P_5 (vertex labels are omitted for simplicity).

(2) Estimate SE-*triples*. With $\mathsf{Nbr}(u_i)$ as the designated endpoints, PathCE next estimates the statistics of $Q_i[\mathsf{Nbr}(u_i)]$ and includes them to $\mathcal{G}$ (line 4). Recall that we require $\mathsf{Nbr}(u_i) \leq 2$ to ensure the statistics remain representable in $\mathcal{G}$. The statistics are computed by combining the relevant SE-triples included in $\mathcal{G}$ (see Sec. 6).

(3) Reduce PSG *query*. PathCE then simplifies $\mathcal{Q}$ by replacing the subquery $Q_i[\mathsf{Nbr}(u_i)]$ with a new edge (line 5). Specifically,

PathCE eliminates u_i and its incident edges from Q and adds a new edge labeled $Q_i[\mathsf{Nbr}(u_i)]$ to connect u_i's neighbors. Here, u_i is referred to as an *eliminated node*. When $\mathsf{Nbr}(u_i)$ consists of a single vertex u', then a dangling edge labeled $Q_i[u']$ is attached to u'. An edge is *dangling* if one of its endpoints links to an eliminated node, *e.g.,* u_i. See Fig. 4(a) for an example, where an eliminated node is depicted with a doted cycle. This step simplifies Q since it decreases the vertex number by one. Also note that the statistics for the new edge labeled $Q_i[\mathsf{Nbr}(u_i)]$ are represented as SE-triples and have been computed in the previous step. Consequently, the estimation iteration continues with the revised Q and updated $\mathcal{G}$.

After $k-1$ iterations, Q consists of only the last vertex u_k. with t dangling edges attached to u_k, where $t \geq 1$. PathCE applies a function AggCnt to aggregate statistics of $Q[u_k]$. Specifically, for each summary vertex V_i with $\mathcal{L}(V_i) = L(u_k)$, AggCnt computes a count c_i to estimate $|\{\rho \in Q(G) \mid \rho(u_k) \in V_i\}|$. The final estimate $|Q(G)|$ is obtained by summing over all such c_i values (line 6). The computation of c_i is performed by exploring the statistics of the dangling edges attached to u_k in Q (see Sec. 6 for details).

Example 6: Figures 4(a)–4(e) demonstrate the estimation process of $|Q(G)|$, using the estimation scheme $(Q, \mathcal{S})$ as depicted in Fig. 3. The subqueries involved in the estimation process are shown in Fig. 4(e). Following the estimation order $\mathcal{S} = [u_9, u_7, u_5, u_3, u_1]$, PathCE conducts the estimation of $|Q(G)|$ as follows.

(1) Note that $\mathsf{Nbr}(u_9) = \{u_5\}$. The first iteration processes the subqery $Q_1[u_5]$ as shown in Fig. 4(e). The SE-triples of $Q_1[u_5]$ are of the form (c, d, ∞). After $Q_1[u_5]$ is removed, a dangling edge labeled $Q_1[u_5]$ is attached to u_5 as shown in Fig. 4(a). In the revised PSG query, the eliminated node u_9 is depicted as a dotted circle.

(2) Similarly, in the second iteration, PathCE processes the subqery $Q_2[u_5]$ and the revised PSG query Q is shown in Fig. 4(b).

(3) In the third iteration, PathCE eliminates u_5 and processes the subqery $Q_3[u_1, u_3]$ with u_1 and u_3 as its designated endpoints (see Fig. 4(e)). In the revised PSG query as shown in Fig. 4(b), we have $\mathsf{Nbr}(u_5) = \{u_1, u_3\}$ and PathCE has all the statistics for each edge of $Q_3[u_1, u_3]$. Indeed, this is because (i) $\mathcal{G}$ initially includes statistics for P_2, P_3 and P_4, and (ii) in previous iterations, PathCE obtains SE-triples for the dangling edges labeled $Q_1[u_5]$ and $Q_2[u_5]$. PathCE thus combines these SE-triples to compute new ones for $Q_3[u_1, u_3]$. Then, PathCE eliminates u_5 and its incident edges from Q and adds a new edge labeled $Q_3[u_1, u_3]$, connecting u_1 and u_3 (see Fig. 4(c)).

(4) In the final iteration, PathCE eliminates u_3 and computes the SE-triples for the subquery $Q_4[u_1]$. Figure 4(d) shows the revised Q with a single vertex u_1 and a dangling edge labeled $Q_4[u_1]$.

In the end, Q reduces to u_1 with a single dangling edge. For each summary vertex V_i such that $\mathcal{L}(V_i) = L(u_1)$, PathCE computes an estimate c_i for $|\{\rho \in Q(G) \mid \rho(u_1) \in V_i\}|$. Since u_1 has a single dangling edge, we let $c_i = c$, where (c, d, ∞) is the SE-triple associated with the summary edge $(V_i, U, Q_4[u_1])$ in $\mathcal{G}$, computed in the fourth iteration. The sum of all c_i is then returned as $|Q(G)|$. □

4.2 Challenges and Approaches

We highlight the challenges in developing the PathCE framework.

Challenge 1: Effective estimation scheme generation. The first question is how to obtain an initial PSG query Q from Q with a minimized number of vertices, as Q's vertex number determines the number of estimation iterations and impacts estimation accuracy significantly. The second question is how to generate an estimation order $\mathcal{S}$ from Q so that the overall estimation process can proceed as a sequence of SE-triple computation. To see the difficulty, consider the case where Q is a 4-clique. Then any estimation order of Q starts with a subquery in which the eliminated node has 3 neighbors. This violates the requirements of the extracted subqueries.

To address the first question, we develop a divide-and-conquer algorithm Dcmp that computes a PSG query Q with the minimal number of vertices for a given query Q (Sec. 5.1). For the second question, we adopt a *minimal neighbors* strategy to generate a feasible estimation order, if possible. When no feasible estimation order can be produced, such as in the 4-clique case, we introduce a *pruning strategy* to remove from Q some edges that have minimal impact on estimation accuracy, creating a new query Q' that allows a feasible elimination order from the PSG query of Q' (Sec. 5.2).

Challenge 2: Cardinality upper bound estimation. Prior studies show that upper bound estimates help generate high-quality query plans [3, 10, 48]. Moreover, upper bound estimation usually does not rely on statistical assumptions and offers theoretical guarantees [1, 2]. The challenge is how to provide upper bound estimates within PathCE, ensuring that each estimate C satisfies $|Q(G)| \leq C$.

Inspired by [1, 3], we have included *maximal-degree* statistics in the SE-triples for path queries and PSG construction. These statistics are then used for estimation within each extracted subquery (Sec. 6). We employ a graph-native way to compute upper bound estimates for each such subquery. By an inductive analysis, we can show that $|Q(G)| \leq C$ (Theorem 3, Sec. 6).

Challenge 3: Scalable PSG **construction**. Given a graph G, a vertex partition Π_V and a path query set $\mathbb{P}$, it is a non-trivial to construct a PSG of G *w.r.t.* Π_V and $\mathbb{P}$ since each SE-triple includes both count and maximal-degree statistics of a path query P. A naive approach is to compute the match set $P(G)$ first and then derive the SE-triples from $P(G)$. However, careful analysis indicates that this would incur significant computational and space overheads.

To address this challenge, we develop a parallelly scalable algorithm PSGBuilder to construct PSG's in linear time *w.r.t.* the input graph size $|G|$. PSGBuilder achieves this by utilizing a compact intermediate representation of SE-triples and conducting the computation in a bottom-up manner. Better still, the algorithm PSGBuilder is *parallelly scalable* [22], *i.e.,* it guarantees to reduce the running time when working with more processors. This implies that PSGBuilder is capable of constructing PSG's for large graphs.

Remark. PathCE is a path-centric cardinality estimator that differs from FactorJoin [48], CEG [5] and GLogS [23] in the followings.

(1) PathCE relies exclusively on statistics of path queries, organized in a PSG graph. When path queries are restricted to single-edge patterns, the statistics used by PSG reduce to those adopted by FactorJoin. In contrast, CEG and GLogS consider statistics of general graph queries beyond path queries, *e.g.,* triangles.

(2) While PathCE and FactorJoin use maximal-degree statistics to

ensure cardinality upper bounds, PathCE adopts an graph-based approach, whereas FactorJoin uses a probabilistic graphical model. Both CEG and GLogS rely on statistical assumptions for estimation and do not guarantee cardinality upper bounds.

(3) PathCE builds statistics in a parallelly scalable manner, making it suitable for large graph datasets. In contrast, FactorJoin and CEG do not include statistics construction algorithms. While GLogS includes its own statistics construction mechanism, it relies on graph sparsification to scale to large graphs.

5 ESTIMATION SCHEME GENERATION

In this section, we show how to generate an estimation scheme for a given query Q. We outline the generation workflow as follows.

Workflow. As outlined in Algorithm 2, PathCE first utilizes a procedure Dcmp to decompose Q into a PSG query $\mathcal{Q}$. The optimization goal of Dcmp is to minimize the number of vertices of $\mathcal{Q}$ to improve estimation accuracy. Next, PathCE tries to compute an estimation order $\mathcal{S}$ from $\mathcal{Q}$ by GenEstOrder (line 1). If that fails, PathCE then tries to prune some edges from Q so that Q has a better chance to generate a feasible estimation order (line 3). After pruning, PathCE retries Dcmp to compute a new $\mathcal{Q}$ and GenEstOrder to find a new $\mathcal{S}$ (line 4). Once a feasible estimation order $\mathcal{S}$ is identified, PathCE returns $(\mathcal{Q}, \mathcal{S})$ as the estimation scheme for Q (line 5).

In the rest of this section, we discuss the ideas behind the procedures Dcmp, GenEstOrder and Prune in more details.

5.1 PSG Query Computation

We first develop an algorithm Dcmp to decompose a given query Q into a PSG query $\mathcal{Q}$ such that $\mathcal{Q}$'s vertex number is minimized. The PSG query $\mathcal{Q}$ of Q satisfies the following requirements.

- **(R0)** The vertex set of $\mathcal{Q}$ is a subset of Q.
- **(R1)** Each edge e in $\mathcal{Q}$ represents a subpath P of Q with statistics included in $\mathcal{G}$. In that case, we say that e *covers* the edges of P.
- **(R2)** Every edge of Q is covered by exactly one edge of $\mathcal{Q}$.
- **(R3)** $\mathcal{Q}$'s vertices can only occur as the endpoints of the subpaths that $\mathcal{Q}$ covers, which is to ensure the estimation correctness.

Note that from (R0)(R1)(R2)(R3), if a vertex u of Q has a degree ≥ 3, then u must be kept in $\mathcal{Q}$. We call each such vertex u a *pivot* of Q. With the pivots, Dcmp obtains a PSG query $\mathcal{Q}$ as outlined below.

Algorithm Dcmp. The algorithm Dcmp first identifies all the pivots of Q. From these pivots, it conducts a series of DFS traversal on Q to compute a set PSet of subpaths of Q. Specifically, starting from a pivot u, a DFS traversal terminates when it either hits a node with degree 1, *i.e.,* the DFS cannot continue, or it hits a vertex with degree ≥ 3, *i.e.,* the starting pivot u or another pivot. Each DFS traversal is included as a path in PSet. It is possible that statistics of a path query P in PSet are *not* in $\mathcal{G}$. By requirement (R1), Dcmp next utilizes a sub-routine PathDP to decompose each such path P in Pset into paths $P_1,\ldots,P_k$ to minimize k so that the statistics of every P_i are included in $\mathcal{G}$. Each P_i and its endpoints are then added as an edge to $\mathcal{Q}$, *i.e.,* these P_i's are used by $\mathcal{Q}$ to cover Q. PathDP adopts dynamic programming to obtain a decomposition of P as follows. Let $P = u_0e_1u_1\ldots u_{n-1}e_nu_n$ be an n-path in PSet and $\mathbb{P}$ be the set of path queries that $\mathcal{G}$ is constructed from. Denote

Algorithm 2: GenScheme: Estimation scheme generation

Input: A graph query Q and a PSG $\mathcal{G}$ *w.r.t.* graph G.
Output: An estimation scheme $(\mathcal{Q}, \mathcal{S})$ for Q.

1 $\mathcal{Q} \leftarrow$ Dcmp$(Q, \mathcal{G})$; $\mathcal{S} \leftarrow$ GenEstOrder$(\mathcal{Q})$;
2 **while** $\mathcal{S} = \emptyset$ **do**
3 $\quad Q \leftarrow$ Prune$(Q, \mathcal{Q})$;
4 $\quad \mathcal{Q} \leftarrow$ Dcmp$(Q, \mathcal{G})$; $\mathcal{S} \leftarrow$ GenEstOrder$(\mathcal{Q})$;
5 **return** $(\mathcal{Q}, \mathcal{S})$;

by $P_{ij} = u_ie_{i+1}u_{i+1}\ldots u_{j-1}e_ju_j$ the subpath of P between v_i and v_j, where $0 \leq i \leq j \leq n$, and define $f(P)$ as the minimum number of paths in $\mathbb{P}$ to decompose P. We give the recurrence as follows:

$$f(P_{ij}) = \begin{cases} 1 & P_{ij} \in \mathbb{P} \\ \min\{f(P_{ik}) + f(P_{kj}) \mid i \leq k \leq j\} & \text{otherwise.} \end{cases} \quad (1)$$

Clearly, this recurrence can be solved using dynamic programming.

The algorithm Dcmp produces an optimal PSG query $\mathcal{Q}$ for a given graph query Q, as summarized in Proposition 1.

Proposition 1: *Given a query Q and $\mathcal{G}$, the algorithm* Dcmp *produces a* PSG *query $\mathcal{Q}$ of Q such that $\mathcal{Q}$ has the minimum number of vertices and meets the requirements of (R0),(R1), (R2) and (R3).* □

5.2 Estimation Order Generation and Pruning

We next discuss how to generate a feasible estimation order for a PSG query $\mathcal{Q}$. Let $\mathcal{S} = [u_1, \ldots, u_k]$ be a vertex order of $\mathcal{Q}$. We say that $\mathcal{S}$ is a *feasible* estimation order for $\mathcal{Q}$ if, in every subquery produced using the scheme $(\mathcal{Q}, \mathcal{S})$, we have $\mathsf{Nbr}(u_i) \leq 2$.

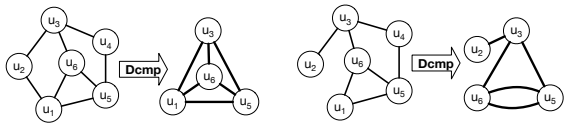
Below, we first introduce the *minimal neighbors strategy* used by GenEstOrder to produce a feasible estimation order $\mathcal{S}$, if such an order exists. Next, we propose a pruning strategy to handle cases where no feasible $\mathcal{S}$ can be generated for $\mathcal{Q}$.

Minimal neighbors strategy. Given $\mathcal{Q}$, GenEstOrder simulates the estimation process and uses a *minimal neighbors* strategy to determine $\mathcal{S}$. In the i-th iteration, GenEstOrder selects a vertex v with the fewest neighbors in $\mathcal{Q}$ as the i-th vertex in $\mathcal{S}$. If multiple vertices have the minimal number of neighbors, it randomly picks one. GenEstOrder then removes u and its incident edges from $\mathcal{Q}$. If u has exactly two neighbors in $\mathcal{Q}$, GenEstOrder also adds a new edge connecting them after removing u, to simulate the estimation process. If u has more than two neighbors, *i.e.,* this would lead to an invalid subquery during estimation, GenEstOrder terminates and the estimation order generation fails. Otherwise, the process continues with the updated $\mathcal{Q}$. If no failure occurs, GenEstOrder returns the vertex sequence $\mathcal{S}$ as the estimation order.

As GenEstOrder simulates the estimation process when generating $\mathcal{S}$, if no failure occurs then the output $\mathcal{S}$ must be feasible for $\mathcal{Q}$. The lemma below shows that (i) GenEstOrder suffices for acyclic queries, and (ii) additional efforts are needed for cyclic queries.

Lemma 2: *(i)* GenEstOrder *always returns a feasible estimation order $\mathcal{S}$ for an acyclic $\mathcal{Q}$. (ii) If* GenEstOrder *fails for a cyclic $\mathcal{Q}$, then any permutation of $\mathcal{Q}$'s vertices is not feasible for $\mathcal{Q}$.* □

Clearly, GenEstOrder fails on the query $\mathcal{Q}$ depicted in Fig. 5(a) and any vertex order of $\mathcal{Q}$ is not a feasible estimation order. In light of Lemma 2, we propose a novel pruning strategy to handle cyclic queries when no feasible estimation order can be found.



(a) Query Q and its PSG query Q (b) Q' and Q' after pruning

Figure 5: An example query that needs pruning

Pruning strategy. If GenEstOrder fails for a PSG query $\boldsymbol{Q}$ of Q, we prune edges from Q to obtain a subquery Q', such that its PSG query $\boldsymbol{Q}'$ has a feasible estimation order $\mathcal{S}'$. Estimation is then performed using $(\boldsymbol{Q}', \mathcal{S}')$. To ensure the result is still an upper bound on $|Q(G)|$, only edges within a cycle of Q are pruned.

To illustrate, consider the query Q in Fig. 5(a), where no feasible estimation order is available for its PSG query $\boldsymbol{Q}$. By pruning the edge (u_1, u_2) from Q, we obtain the subquery Q', as shown in Fig. 5(b). Assuming $\mathcal{G}$ includes statistics for all path queries of length up to 2, the new PSG query $\boldsymbol{Q}'$ for Q' is shown in Fig. 5(b), and $\mathcal{S}' = [u_2, u_3, u_6, u_5]$ is a feasible estimation order for $\boldsymbol{Q}'$. Let C be the estimated cardinality using $(\boldsymbol{Q}', \mathcal{S}')$. As will be shown in Sec. 6, $|Q'(G)| \leq C$, which means C remains an upper bound on $|Q(G)|$ since $|Q(G)| \leq |Q'(G)| \leq C$. The pruned edge (u_1, u_2) lies within a cycle of Q, so any match for Q is also a match for Q'.

6 SUBQUERY ESTIMATION

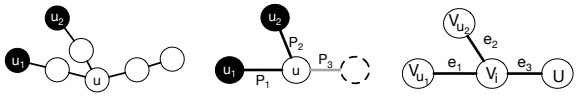
In this section, we show how to estimate the statistics for a subquery Q extracted in an estimation iteration.

We focus on the case where Q has two designated endpoints; the case with only one designated endpoint can be treated as a special case. Let u be the eliminated node, and let u_1 and u_2 be its neighbors, which serve as the designated endpoints of Q_i (see Fig. 6(a)). The goal is to compute the statistics for $Q[u_1, u_2]$, based on the statistics included in $\mathcal{G}$. Specifically, for each summary vertex pair (V_{u_1}, V_{u_2}) that matches (u_1, u_2), we compute an SE-triple (x, y, z) to represent the summary of $Q[u_1, u_2]$ with respect to (V_{u_1}, V_{u_2}).

We compute (x, y, z) using *maximal-degree statistics*, inspired by prior studies [3, 48]. Unlike the bound formula generator in BoundSketch [3] or the probabilistic graph models in FactorJoin [48], our graph-based approach consists of two steps: PSG *query match* and SE-*triple estimation*. Our approach is based on the observation that *a query on the a data graph are accurate while the matches on the* PSG *graph should represent estimated results.*

(1) PSG **query match**. This step is to extract SE-triples encoded in $\mathcal{G}$ for the estimation in the second step. Let $\boldsymbol{Q}[u_1, u_2]$ be the corresponding PSG query of $Q[u_1, u_2]$. Intuitively, each edge of $\boldsymbol{Q}[u_1, u_2]$ represents a sub-path of $Q[u_1, u_2]$ (see Fig. 6 for an example). We first run the PSG query $\boldsymbol{Q}[u_1, u_2]$ *on* PSG *graph* $\mathcal{G}$ and require that u_1 is matched to V_{u_1} and u_2 is matched to V_{u_2}. This is well-defined since $\mathcal{G}$ is also a graph by design. Note that $Q[u_1, u_2]$ may have dummy nodes, *i.e.,* nodes eliminated during estimation (see the dotted circles in Fig. 6(b)). We require dummy nodes to match the dummy summary vertex U of $\mathcal{G}$.

By the definition of PSG, there are exactly M matches of $\boldsymbol{Q}[u_1, u_2]$ in $\mathcal{G}$. Specifically, let $V_1, \ldots V_M$ be the summary vertices matching u. Each match can be represented as a subgraph $\mathcal{G}_i$ of $\mathcal{G}$, where node u is mapped to V_i in $\mathcal{G}_i$. The vertex set of $\mathcal{G}_i$ includes V_{u_1}, V_{u_2} and V_i, and U (if $\boldsymbol{Q}[u_1, u_2]$ contains an eliminated node). For the query $\boldsymbol{Q}[u_1, u_2]$ as shown in Fig. 6(b), a match $\mathcal{G}_i$ is shown in Fig. 6(c). The label of each summary edge in $\mathcal{G}_i$ also matches that in $\boldsymbol{Q}[u_1, u_2]$. For example, the summary edge e_1 of $\mathcal{G}_i$ has label



(a) $Q[u_2,u_2]$ (b) $Q[u_2,u_2]$ (c) $\mathcal{G}_i$

Figure 6: Query $Q[u_1, u_2]$ and its PSG version $\boldsymbol{Q}[u_1, u_2]$, along with a subgraph $\mathcal{G}_i$ of $\mathcal{G}$ that matches $\boldsymbol{Q}[u_1, u_2]$.

P_1, matching the edge between u_1 and u.

Fix a PSG subgraph $\mathcal{G}_i$ and let $\mathcal{G}'$ be a connected subgraph of $\mathcal{G}_i$. Let Q' be the query formed by connecting the subqueries encoded as edge labels in $\mathcal{G}'$. Note that , Q' must be a subquery of Q, and we have $Q' = Q$ when $\mathcal{G}' = \mathcal{G}_i$. Along the same lines as Definition 3.1, we define the following for each subgraph $\mathcal{G}'$ of $\mathcal{G}_i$:

$$c(\mathcal{G}') = \big|\{\rho \in Q'(G) \mid \rho(u_1) \in V_{u_1}, \rho(u_2) \in V_{u_2}, \rho(u) \in V_i\}\big|,$$
$$d_1(\mathcal{G}') = \max_{v_1 \in V_{u_1}} \big|\{\rho \in Q'(G) \mid \rho(u_1) = v_1, \rho(u_2) \in V_{u_2}, \rho(u) \in V_i\}\big|,$$
$$d_2(\mathcal{G}') = \max_{v_2 \in V_{u_2}} \big|\{\rho \in Q'(G) \mid \rho(u_1) \in V_{u_1}, \rho(u_2) = v_2, \rho(u) \in V_i\}\big|.$$
$$d(\mathcal{G}') = \max_{v \in V_i} \big|\{\rho \in Q'(G) \mid \rho(u_1) \in V_{u_1}, \rho(u_2) \in V_{u_2}, \rho(u) = v\}\big|.$$

Then, by Definition 3.1 and the observation that $\{V_1, \ldots, V_M\}$ represents a partition of the vertices matching u, we must have

$$(c, d_1, d_2) = \sum \Big\{ (c(\mathcal{G}_i), d_1(\mathcal{G}_i), d_2(\mathcal{G}_i)) \;\Big|\; i = 1, \ldots, M \Big\},$$

where (c, d_1, d_2) is the summary of $Q[u_1, u_2]$ *w.r.t.* (V_{u_1}, V_{u_2}).

(2) SE-**triple estimation**. For each PSG subgraph $\mathcal{G}_i$, we first compute a triple (x_i, y_i, z_i) such that $c(\mathcal{G}_i) \leq x_i$, $d_1(\mathcal{G}_i) \leq y_i$, and $d_2(\mathcal{G}_i) \leq z_i$. We then let $(x, y, z) = \sum_{i=1}^{M} (x_i, y_i, z_i)$ be the estimate for (c, d_1, d_2). The following theorem is immediate.

Theorem 3: (1) $(c, d_1, d_2) \leq (x, y, z)$. **(2)** *The estimated cardinality returned by Algorithm 1 is an upper bound on the true cardinality.* □

Fix a PSG subgraph $\mathcal{G}_i$, we next compute a triple (x_i, y_i, z_i) that upper bounds $(c(\mathcal{G}_i), d_1(\mathcal{G}_i), d_2(\mathcal{G}_i))$, which can be done by leveraging the maximal-degree statistics, as explored by prior studies [3]. To simplify our discussion, let us consider the $\mathcal{G}_i$ as shown in Fig. 6(c). Let $\mathcal{G}_{e_1}$, $\mathcal{G}_{e_2}$, and $\mathcal{G}_{e_3}$ be the subgraphs of $\mathcal{G}_i$ consisting of the single edges e_1, e_2, and e_3; and $\mathcal{G}'_{e_1}$, $\mathcal{G}'_{e_2}$, and $\mathcal{G}'_{e_3}$ the PSG subgraphs obtained by pruning e_1, e_2, and e_3 from $\mathcal{G}_i$, respectively.

(i) Bounds on maximal-degree statistics can be obtained in a bottom-up manner. For example, for $d_1(\mathcal{G}_i)$, $d_2(\mathcal{G}_i)$, and $d(\mathcal{G}_i)$, we have

$$d_1(\mathcal{G}_i) \leq d_1(\mathcal{G}_{e_1}) \cdot d(\mathcal{G}'_{e_1}), \quad d_2(\mathcal{G}_i) \leq d_2(\mathcal{G}_{e_2}) \cdot d(\mathcal{G}'_{e_2}),$$
$$d(\mathcal{G}_i) \leq \min\{d(\mathcal{G}_{e_1}) \cdot d(\mathcal{G}'_{e_1}),\ d(\mathcal{G}_{e_2}) \cdot d(\mathcal{G}'_{e_2}),\ d(\mathcal{G}_{e_3}) \cdot d(\mathcal{G}'_{e_3})\}.$$

Observe that each graph on the RHS is a subgraph of the one on the LHS. By replacing "≤" with "=", we can compute upper bounds for $d_1(\mathcal{G}')$, $d_2(\mathcal{G}')$, and $d(\mathcal{G}')$ for each subgraph $\mathcal{G}'$ of $\mathcal{G}_i$. We then set y_i and z_i to be the upper bounds on $d_1(\mathcal{G}_i)$ and $d_2(\mathcal{G}_i)$, respectively.

(ii) Observe that $c(\mathcal{G}_i) \leq c(\mathcal{G}_{e_1}) \cdot d(\mathcal{G}'_{e_1})$. Indeed, (a) there are at most $c(\mathcal{G}_{e_1})$ matches of P_1 such that u_1 is mapped a vertex in V_{u_1} and u is mapped to a vertex in V_i; (b) and a match can be extended to at most $d(\mathcal{G}'_{e_1})$ matches of Q. Similarly, $c(\mathcal{G}_i) \leq c(\mathcal{G}'_{e_1}) d(\mathcal{G}_{e_1})$. By considering all three edges e_1, e_2 and e_3 of $\mathcal{G}_i$, we have

$$c(\mathcal{G}_i) \leq \min\{c(\mathcal{G}_{e_1}) \cdot d(\mathcal{G}'_{e_1}),\ c(\mathcal{G}_{e_2}) \cdot d(\mathcal{G}'_{e_2}),\ c(\mathcal{G}_{e_3}) \cdot d(\mathcal{G}'_{e_3})\},$$
$$c(\mathcal{G}_i) \leq \min\{c(\mathcal{G}'_{e_1}) \cdot d(\mathcal{G}_{e_1}),\ c(\mathcal{G}'_{e_2}) \cdot d(\mathcal{G}_{e_2}),\ c(\mathcal{G}'_{e_3}) \cdot d(\mathcal{G}_{e_3})\}.$$

By replacing "≤" with "=" and applying a bottom-up computation, we can derive an upper bound for $c(\mathcal{G}_i)$ and set it as x_i.

7 SCALABLE PSG CONSTRUCTION

We develop a scalable algorithm PSGBuilder for PSG construction.

We first construct a path query set $\mathbb{P}$ for a given graph $G = (V, E, L)$. PSGBuilder enumerates all possible k-path queries for $k \leq K$, by utilizing a graph $G_S = (V_S, E_S, L_S)$ that specifies the the valid vertex types and edges types of G [29]. More specifically, G_S is also a labeled graph satisfying the following: (a) $L_S(u_1) \neq L_S(u_2)$ for $u_1 \neq u_2$, and (b) for each edge (u_1, u_2, ℓ) in E, there is an edge (u_1', u_2', ℓ) in E_S such that $L(u_1) = L_S(u_1')$ and $L(u_2) = L_S(u_2')$. PSGBuilder conducts a series of k-walks on G_S and includes them as k-path queries in $\mathbb{P}$, with duplicate k-path queries omitted.

Challenges. Given a graph G, a graph summary vertex partition Π_V and a path query set $\mathbb{P}$, PSGBuilder next constructs a PSG of G *w.r.t.* Π_V and $\mathbb{P}$. Let P be a path query in $\mathbb{P}$. The goal is to compute all summary edges and their associated SE-triples for P. Recall that PSGBuilder must also compute maximal-degree statistics, which is more challenging. A straightforward approach is to first compute $P(G)$ and then derive the SE-triples. This requires $O(k|E|)$ space and takes $O(|E|^k)$ time, since obtaining the maximal-degree statistics mandates scanning the entire $P(G)$. To address these challenges, PSGBuilder utilizes a compact intermediate representation of SE-triples and computes them in linear time *w.r.t.* $|G|$. Furthermore, the computation is *parallelly scalable* [22], *i.e.,* PSGBuilder guarantees to reduce the running time when working with more processors.

Compact representation. Consider a path query P in $\mathbb{P}$ with h_P and t_P as it head and tail vertices. Let $V_h^1, \ldots, V_h^M$ and $V_t^1, \ldots, V_t^M$ be the summary vertices in $\mathcal{V}$ with label $L(h_P)$ and $L(t_P)$, respectively. For each vertex v in $\bigcup_{i=1}^{M} V_h^i$, we associate with v a head vector $\mathcal{H}_v^P$ of size M, where the i-th component $\mathcal{H}_v^P[i]$ records the number of matches ρ of P such that $\rho(h_P) = v$ and $\rho(t_P)$ lies in the summary vertex V_t^i. Similarly, we associate with each vertex u in $\bigcup_{i=1}^{M} V_t^i$ a tail vector $\mathcal{T}_u^P$, where the i-th component $\mathcal{T}_u^P[i]$ tracks the number of matches ρ of P such that $\rho(t_P) = u$ and $\rho(h_P)$ belongs V_h^i.

The benefits of using head and tail vectors are twofold.

(**1**) We can compute the SE-triples for path query P by aggregating these vectors. Indeed, the SE-triple (c, d_1, d_2) associated with the summary edge (V_h^i, V_t^j, P) can be computed by

$$c = \sum \{\mathcal{H}_v^P[j] \mid v \in V_h^i\} = \sum \{\mathcal{T}_u^P[i] \mid u \in V_t^j\}, \quad (2)$$

$$d_1 = \max\{\mathcal{H}_v^P[j] \mid v \in V_h^i\}, \quad d_2 = \max\{\mathcal{T}_u^P[i] \mid u \in V_t^j\}. \quad (3)$$

(**2**) We can compute the head and tail vectors in a bottom-up manner, *without* first computing and storing the path matches. Let e_0 and e_1 be the edges attached to h_P and t_P, and P_0 and P_1 be the path queries obtained by removing e_0 and e_1 from P, respectively. Let v be a vertex in $\bigcup_{i=1}^{M} V_h^i$ and u be a vertex in $\bigcup_{i=1}^{M} V_t^i$. We can compute $\mathcal{H}_v^P$ and $\mathcal{T}_u^P$ from the vectors related to P_0 and P_1 by

$$\mathcal{H}_v^P = \sum \{\mathcal{H}_w^{P_0} \mid w \in \mathsf{Nbr}(v) \wedge \mathsf{Match}(v, w, e_0)\}, \quad (4)$$

$$\mathcal{T}_u^P = \sum \{\mathcal{T}_{w'}^{P_1} \mid w' \in \mathsf{Nbr}(u) \wedge \mathsf{Match}(w', u, e_1)\}. \quad (5)$$

Here, $\mathsf{Nbr}(v)$ records the neighbors of v in G, $\sum$ denotes pairwise summation over vector components, and $\mathsf{Match}(v, w, e_0)$ returns true if there is an edge from v to w that matches e_0, similarly for $\mathsf{Nbr}(u)$ and $\mathsf{Match}(w', u, e_1)$. Both P_0 and P_1 are shorter than P, allowing the head and tail vectors to be computed recursively.

Parallelly scalable processing. PSGBuilder computes the head and tail vectors, as well as the SE-triples, recursively in a parallelly scalable manner. Specifically, let $\mathbb{P}_i$ be the set of path queries in $\mathbb{P}$ of length i. In the i-th iteration, PSGBuilder computes all head and tail vectors for path queries in $\mathbb{P}_i$ using Equations (4) and (5). From these vectors, it applies Equations (2) and (3) to obtain all SE-triples for $\mathbb{P}_i$. Observe that the computation of head and tail vectors can be parallelized at the granularity of vertices of G, while SE-triple computation can be parallelized on the basis of summary vertex pairs.

Analysis. The running time is dominated by the computation of SE-triples, which is bounded by $O\left(\frac{1}{C}NM(|E| + |V|)\right)$. Here, C is the number of processors and N is the number of k-path queries in $\mathbb{P}$, bounded by $(|V_S| + |E_S|)^K$. Indeed, PSGBuilder requires $O(M|E|)$ time to process the head and tail vectors for each path query P in $\mathbb{P}$, and $O(M|V|)$ time to compute the SE-triples for P. Note that N depends on the vertex types and edge patterns in G, rather than the size of the data graph G. The $\frac{1}{C}$ factor reflects the effective parallelization of the computation. The overall space cost of the algorithm is $O(NM|V| + NM^2)$. This includes $O(NM|V|)$ space for head and tail vectors and $O(NM^2)$ for the SE-triples. By scheduling the processing of head and tail vectors in a DFS fashion [4], the space cost can be reduced to $O(KM|V| + NM^2)$.

Remark. Computing head and tail vectors is essential for PSG construction, which heavily relies on neighborhood access (see Equations (4) & (5)). Graph systems inherently support efficient neighborhood access, thus enable an efficient implementation of PSGBuilder.

8 EVALUATION

Experimental settings. We start with experimental settings.

Datasets and Queries. We use three well-established datasets and workloads: LDBC [8], IMDB [25], and AIDS [37].

(1) LDBC refers to a set of generated datasets from the Linked Data Benchmark Council's Social Network Benchmark, designed for evaluating GDBMSs. We denote LDBC datasets of scale factors 0.1, 0.3, 1, 3 and 10 by LDBC-0.1, LDBC-0.3, LDBC-1, LDBC-3, and LDBC-10, respectively. Unless specified, LDBC refers to LDBC-1. The queries on LDBC consist of 9 queries from the LSQB subgraph query benchmark (Q1–Q9 in [29]) and 11 GLogS queries (marked Q10–Q20, originally P1–P11 in [23]). We remove the properties from the LDBC datasets. Similarly, for all LDBC queries, we remove predicates and convert optional or negative edges into normal edges.

(2) IMDB is a real-world relational dataset containing 21 tables and around 10 million rows. It has been widely used with the Join Order Benchmark (JOB) [5, 25]. Following [42], we convert IMDB into a labeled graph with 12 vertex labels and 13 edge labels. We use the JOB queries (referred to as IMDB queries hereafter) [25], consisting of 33 SQL templates. These query templates are converted into graph queries by retaining only the join query graph topology while omitting predicates and aggregations [42]. All IMDB queries are acyclic.

(3) AIDS is a real-world dataset widely used for subgraph match-

Table 2: Dataset statistics: $\#(L_v)$, $\#(L_e)$, $\#(Q)$ and $|\mathbb{P}|$ indicate the number of vertex labels, edge labels, queries evaluated, path queries for PSG construction, respectively.

Datasets	$\|V\|$	$\|E\|$	$\#(L_v)$	$\#(L_e)$	$\#(Q)$	$\|\mathbb{P}\|$
LDBC	3.73M	21.4M	11	25	20	1243
IMDB	52.6M	119M	12	13	33	471
AIDS	254K	548K	50	4	484	297

ing [19, 33, 37], consisting of 43,905 chemical molecules. We use 484 queries provided by the subgraph cardinality estimation benchmark tool G-CARE [33], which includes 3-, 6-, and 9-path queries, as well as randomly generated star, tree, and generic graph queries.

The statistics of the datasets are summarized in Tab. 2. Note the size $(|V|, |E|)$ for LDBC varies with the scale factor, ranging from $(0.45M, 2.18M)$ for LDBC-0.1 to $(33.9M, 211M)$ for LDBC-10.

Baselines. We prototype PathCE as a Rust library, built from scratch, including the implementation of cardinality estimation and parallel PSG construction. PathCE adopts the GBSA algorithm developed in [48] to generate summary vertex partitions. GBSA is a greedy partitioning heuristic that divides vertices with label ℓ by minimizing the variance in their participation across path queries in $\mathbb{P}$ that have endpoints labeled ℓ. We follow the same setting as FactorJoin and sets $M = 200$ for fair comparison. For each dataset, PathCE sets $K = 3$ to collect all possible k-path queries for $k \leq K$. The number of path queries used for PSG construction for each dataset is summarized in Table 2. We compare PathCE with seven state-of-the-art cardinality estimators, including five summary-based methods: GLogS [23], CEG [5], FactorJoin [48], SumRDF [39] and Color [9], a sampling-based estimator WJ [26], and a ML-based estimator GNCE [35]. **(1)** GLogS [23] is an estimator whose summary records the counts of small patterns, not limited to paths. **(2)** CEG [5] is an estimation framework consisting of nine estimators. We use the estimator that achieves the most accurate estimation. We collect statistics for small patterns with at most 3 vertices for both GLogS and CEG. **(3)** FactorJoin [48][2] is an estimator based on the maximal-degree statistics of 1-path queries, *i.e.,* single edge queries. We set the number of bins to 200 for FactorJoin to match the parameter M in PathCE. **(4)** SumRDF [39] is a summary-based estimator designed for RDF graphs. **(5)** WJ [26] is a sampling-based estimator. For both SumRDF and WJ, we use the implementations and default configurations provided by G-CARE, *e.g.,* the sample ratio of WJ is set to 3%. **(6)** GNCE [35] is an ML-based estimator for knowledge graphs that leverages graph embeddings and GNNs. For offline training of GNCE, we sample 5000 path queries and 5000 star queries from each dataset. **(7)** Color [9] is a summary-based estimator utilizing graph coloring. We use its AvgMix32 variant, the default with the best estimation accuracy.

Environment. We conduct all the experiments on a Linux server with 32 physical cores and 256 GB RAM. For offline model training and cardinality estimation for GNCE, we use an NVIDIA RTX 4070 GPU. We use 32 threads for all baselines during summary construction and query execution, unless stated otherwise. Each experiment is run five times, and the average is reported.

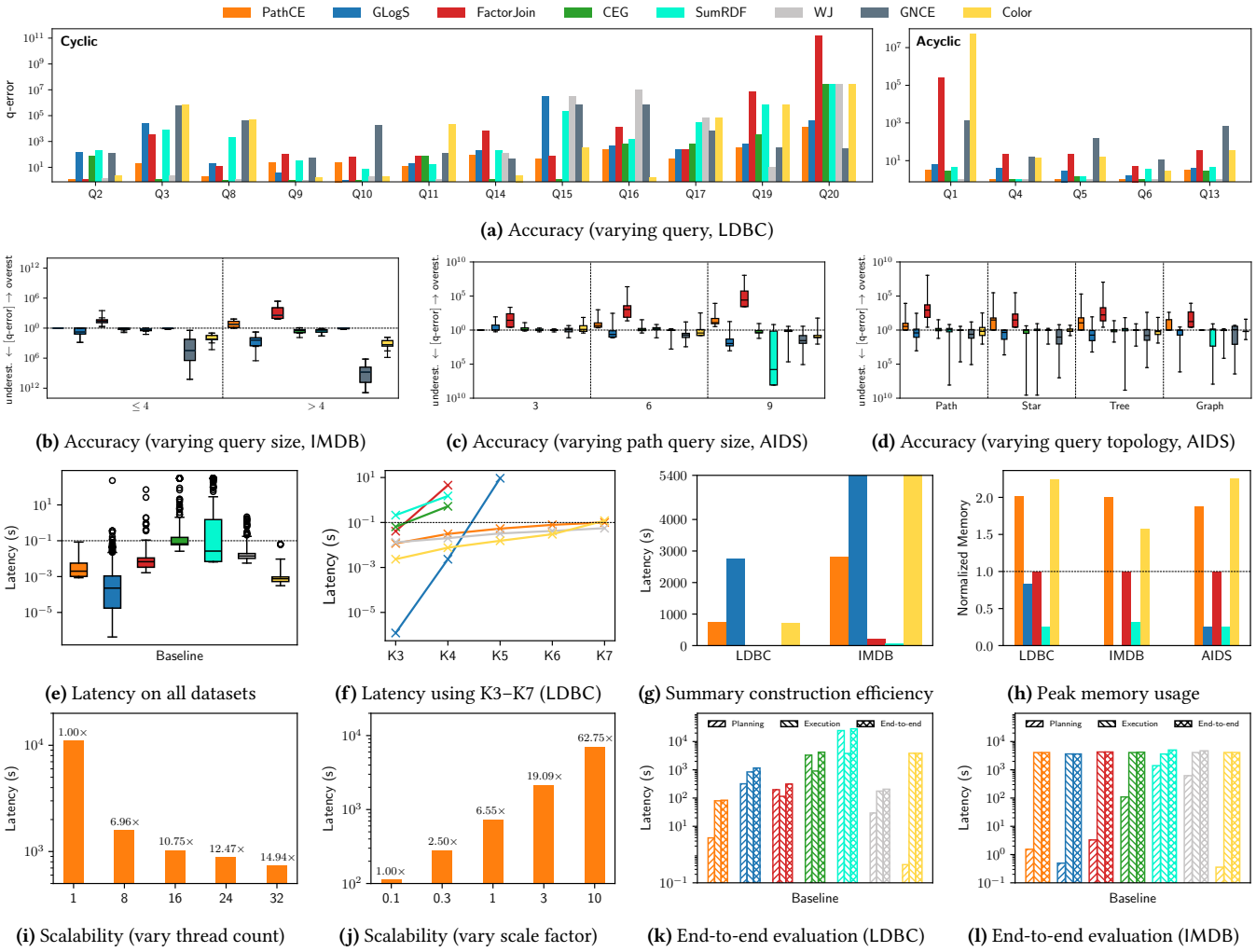
Exp-1: Estimation accuracy. We first evaluate the estimation accuracy of PathCE on all datasets[3]. We use the classical q-error [30] as the metric to evaluate estimation accuracy, defined as $\max\{\frac{e}{c}, \frac{c}{e}\}$, where e is the estimated cardinality for a given query Q, and c is the true cardinality. A lower q-error indicates higher estimation accuracy. Following the common practice of prior studies [33, 48], (a) each estimator is evaluated with its designed summary or ML model; (b) we set $e = 1$ when a baseline fails to produce an estimate for Q within 300 seconds. We find the following.

(1) Varying query on LDBC. Figure 7(a) depicts the q-error of all tested queries on LDBC, where results are grouped as acyclic and cyclic queries. Note that Q7,Q12,Q18 are omitted in Fig. 7(a) since they are identical to Q4, Q2 and Q3 with predicates removed. (**a**) For acyclic LDBC queries, all summary-based estimators, except FactorJoin and Color, perform consistently well, with an average q-error below 5. (**b**) For cyclic LDBC queries, PathCE consistently outperforms FactorJoin and SumRDF. Note that PathCE, FactorJoin and SumRDF utilize path queries of lengths up to 3, 1 and 1, respectively. PathCE also are more accurate than Color on most cyclic queries. These confirm the effectiveness of using longer path queries to reduce estimation iterations. PathCE also shows better accuracy than GLogS and CEG, except for queries including triangles as subqueries, *e.g., Q*3 and *Q*8. As will be highlighted in Exp-3, collecting triangle statistics is cost-prohibitive. (**c**) WJ produces accurate estimates for acyclic queries and small cyclic queries such as Q2 and Q3. However, WJ suffers from severe underestimation on Q15, Q16, Q17, and Q20, which contain large cycles as subqueries. This is primarily due to its sample-and-validate mechanism[26], which makes it prone to sampling failures on cyclic queries, especially those involving large cycles like Q15. In contrast, for acyclic queries and simple cyclic patterns, *e.g.,* Q2 and Q3, WJ is more likely to obtain valid samples, leading to relatively accurate estimates. Similar behavior is also observed on other tested datasets. (**d**) GNCE in general produces less accurate estimates than others because GNCE is trained using sampled path and star queries, while the evaluated LDBC queries are mostly generic graph queries.

(2) Varying query size. Varying the query sizes of IMDB queries and AIDS path queries, Fig. 7(a) and Fig. 7(c) report the accuracy results. Here, the query size is defined as the number of edges in the query graph. Following an approach adopted by G-CARE [33], we reorder the estimates from the least accurate underestimation to the least accurate overestimation before generating the box plot. We observe the following. (**a**) On both datasets, PathCE, CEG, and WJ are the most accurate estimators. As the query size increases, the q-error of PathCE also increases but at a slower rate compared to FactorJoin and SumRDF. On AIDS, the average q-error of PathCE grows from 1 to 313, varying from 3-path queries to 9-path queries. (**b**) While PathCE consistently produces upper bound estimates, Color tends to underestimate most queries on IMDB and AIDS. Both CEG and GLogS shift from overestimation to underestimation as the path query size increases on AIDS, primarily due to their reliance on independence and uniformity assumptions. (**c**) GNCE is less accurate on IMDB than other baselines, similar to the case of

[2]The original implementation of FactorJoin hardcodes the configurations of specific datasets and queries. We re-implemented FactorJoin in Rust based on its paper [48] and verified that it achieves equivalent accuracy on STATS-CEB as the original version.

[3]The true and estimated cardinalities for all baselines can be found in [46].



(a) Accuracy (varying query, LDBC)

(b) Accuracy (varying query size, IMDB)

(c) Accuracy (varying path query size, AIDS)

(d) Accuracy (varying query topology, AIDS)

(e) Latency on all datasets

(f) Latency using K3–K7 (LDBC)

(g) Summary construction efficiency

(h) Peak memory usage

(i) Scalability (vary thread count)

(j) Scalability (vary scale factor)

(k) End-to-end evaluation (LDBC)

(l) End-to-end evaluation (IMDB)

Figure 7: Performance Evaluation

LDBC. In contrast, GNCE has quite decent accuracy on AIDS. This is because we only evaluate path queries on AIDS in this setting.

(3) Varying query topology. We divide all the AIDS queries into four topology categories: path, star, tree, and graph, and evaluate the q-error for all baselines. As shown in Fig. 7(d), PathCE, along with GLogS and CEG, performs consistently well across different query topologies. The key difference is that PathCE always produces upper bound estimates, whereas GLogS and CEG tend to underestimate. SumRDF shows good accuracy for path and star queries but suffers from significant accuracy degradation for other query types. WJ produces the most accurate estimates but tends to underestimate due to sampling failures for complex graph queries.

Exp-2: Estimation latency. We compare the estimation latency of all estimators on the LDBC, IMDB, and AIDS datasets. For each query Q, we record the time taken by each estimator to produce an estimate. As in Exp-1, the estimation timeout is set to 300 seconds. The results are shown in Fig. 7(e), where GNCE is omitted since it uses GPU for estimation, while others are CPU-based. In Fig. 7(e), the upper whisker is set as 20 times the IQR from the upper quartile.

(1) PathCE has the lowest average latency among all summary-based competitors, except for Color, which achieves the best estimation latency across nearly all cases. This is primarily due to its use of a topological estimation order, similar to PathCE, along with inference optimizations such as partial aggregation [9]. PathCE is also significantly more efficient than the sampling-based WJ, as expected. On LDBC, PathCE achieves an average latency of 0.02 seconds, significantly lower than GLogS, FactorJoin, CEG, SumRDF, and WJ, which take 13.43, 6.24, 24.57, 35.75, and 0.09 seconds, respectively. A similar trend is also observed on IMDB and AIDS.

(2) PathCE's stability is further highlighted by its low variance, even on cyclic LDBC queries. While GLogS achieves lower latency for most queries compared to PathCE, it struggles with complex queries, *e.g.*, 227.02 seconds for LDBC Q20, which contains 3 diamond subqueries. CEG faces similar challenges as GLogS, mainly due to the exponential search space during estimation. PathCE avoids this through an effective estimation scheme generation, assisted by pruning techniques (Sec. 5.2). In contrast, CEG's high latency limits its practical use, despite its good estimation accuracy. This becomes more evident in the end-to-end tests (see Exp-5).

(3) To explore further, we handcraft five additional clique queries (K3 to K7) using person nodes and knows edges from the LDBC dataset. As shown in Fig. 7(f), PathCE remains stable with latencies under the practical threshold 0.1 seconds, showing only slight increases as clique size grows. Color also performs consistently well as PathCE. Other estimators, however, exhibit significant latency spikes. FactorJoin, CEG and SumRDF timeout from K5, and GLogS fails for K6 and K7. Although WJ achieves similar performance as PathCE, it encounters underestimation due to sampling failure starting from K4. PathCE's superior performance verifies the effectiveness of Dcmp algorithm and pruning techniques.

Exp-3: Summary construction efficiency. The efficiency of summary construction is critical for real-world deployment, as discussed in [3, 10, 33, 48]. We measure *(a) construction latency*, *(b) peak memory usage*, and *(c) on-disk storage requirements* for all summary-based estimators except CEG. We exclude CEG as it constructs summaries tailored only for each tested query. We find the following.

(1) Construction latency. As shown in Fig.7(g), PathCE constructs its summary on LDBC in 734 seconds, significantly faster than GLogS (2758 seconds), as the latter also collects triangle counts. The performance gap is even more pronounced on IMDB, where PathCE completes the construction in 2831 seconds, while GLogS fails to finish within 1.5 hours since GLogS needs to enumerate a large number of matches during summary construction. In contrast, PathCE uses a compact representation of intermediate matches, as discussed in Sec. 7, which is much more computationally efficient. FactorJoin and SumRDF have the shortest construction time by using solely single-edge statistics. While PathCE and Color show comparable performance on LDBC, Color suffers from significant slowdown on IMDB, taking over 1.5 hours to complete summary construction despite collecting only single-edge statistics. Given that IMDB is 6.8× larger than LDBC, this inefficiency is likely due to the single-threaded implementation of Color, which limits its scalability on large datasets. For AIDS, since it is much smaller than LDBC and IMDB, all baselines are able to complete summary construction within 60 seconds.

(2) Peak memory usage. Figure 7(h) reports the peak memory usage of all baselines. All values are normalized with FactorJoin treated as the baseline, *i.e.,* set to 1. PathCE incurs higher memory overhead than other summary-based estimators due to two main factors: (a) it maintains more intermediate results to compute both path counts and maximal-degree statistics, and (b) it stores both head and tail vertices in memory during summary construction (see Sec. 7).

(3) On-disk storage size. PathCE requires more disk space compared to FactorJoin and GLogS, *i.e.,* 1145.0 MB, 428.5 MB and 163.3 MB for LDBC, IMDB and AIDS, respectively. This storage requirement is still practically feasible. As discussed in Sec. 3, the PSG storage size of PathCE is independent *w.r.t.* the data graph size. Moreover, a detailed analysis of the estimation logs reveals that on LDBC, only 68 out of 1243 collected path queries are used in cardinality estimation across our entire evaluation. This indicates that the storage overhead can be reduced by 94.5% through pruning unused path queries based on workload coverage. Similarly, on IMDB, storage can be reduced by 91.3%. In contrast, baselines like FactorJoin rely on single-edge queries and thus use nearly all collected statistics.

Exp-4: Scalability of summary construction. We next evaluate the scalability of PathCE, by varying thread counts and graph sizes. The results are shown in Fig. 7(i) and Fig. 7(j). **(1)** As shown in Fig. 7(i), PathCE achieves a 14.95× speedup when varying thread counts from 1 to 32 on dataset LDBC. **(2)** With LDBC datasets from LDBC-0.1 to LDBC-10, Fig. 7(j) reports PathCE's summary construction latency which grows almost linearly, consistent with its time complexity (Sec. 7). The storage size remains stable, 1145.0 MB (not shown), as it is determined by the vertex/edge types of the data graph. These results confirm PathCE's effective scaling with both graph size and number of threads during summary construction.

Exp-5: End-to-end performance. Following prior work [3, 10, 48], we evaluate the end-to-end query latency to assess the impact of PathCE's estimation accuracy on query optimization. We integrate all evaluated estimators[4] into the bottom-up optimizer of the GLogS system [23], under a simplified abstraction: during plan enumeration, the optimizer issues cardinality requests for subqueries, and the estimator returns cardinality estimates. This abstraction allows decoupling estimation from plan enumeration, and has also been adopted by top-down optimizers, such as GOpt [28]. The results for LDBC and IMDB, including planning time, execution time and end-to-end time are shown in Fig. 7(k) and Fig. 7(l), respectively.

(1) PathCE beats all other baselines by 2.46× ∼ 336.71× on LDBC in end-to-end time. PathCE is able to generate the best query plans especially for complex queries like Q19 and Q20 thanks to its accurate upper bound estimates. Thus, PathCE beats all baselines by 1.44× ∼ 48.11× in execution time. PathCE also requires less planning time than all baselines except Color, which, while fastest in planning, suffers from suboptimal plans. Compared to FactorJoin, PathCE achieves a 3.73× speedup in E2E time, with 50.85× and 1.44× improvements in planning and query execution, respectively.

(2) On IMDB, all baselines show similar execution performance, as the queries are acyclic and simple. However, PathCE achieves 2.14× ∼ 911.63× lower planning latency than FactorJoin, CEG, SumRDF, and WJ, and remains comparable to GLogS and Color (gap < 1.5s, negligible relative to end-to-end time).

(3) We also conduct a case study on Q19. Regarding planning, PathCE performs only slightly slower than GLogS, but much faster than other baselines. For query execution, PathCE, CEG and WJ share the best plan. FactorJoin's plan differs slightly but does not scale as well as PathCE. Unlike other estimators, GLogS and Color produce underestimations for Q19, resulting in suboptimal expansion-only plans with large intermediate results. The upper bound estimation of PathCE and FactorJoin helps prevent this issue. CEG and WJ do not face the issue due to more accurate estimation.

9 CONCLUSION

We propose PathCE, a path-centric cardinality estimation framework for subgraph matching. An interesting future direction is to apply PathCE to relational DBMSs. Another direction is to explore how to effectively maintain a PSG in response to graph updates.

[4] GNCE is omitted as it uses GPU for estimation.

REFERENCES

[1] Mahmoud Abo Khamis, Hung Q. Ngo, and Dan Suciu. 2017. What Do Shannon-type Inequalities, Submodular Width, and Disjunctive Datalog Have to Do with One Another? *(SIGMOD '17)*. ACM, Chicago Illinois USA, 429–444.

[2] Albert Atserias, Martin Grohe, and Dániel Marx. 2008. Size Bounds and Query Plans for Relational Joins. In *2008 49th Annual IEEE Symposium on Foundations of Computer Science*. IEEE, Philadelphia, PA, USA, 739–748.

[3] Walter Cai, Magdalena Balazinska, and Dan Suciu. 2019. Pessimistic Cardinality Estimation: Tighter Upper Bounds for Intermediate Join Cardinalities *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA.

[4] Hongzhi Chen, Miao Liu, Yunjian Zhao, Xiao Yan, Da Yan, and James Cheng. 2018. G-Miner: an efficient task-oriented graph mining system. In *Proceedings of the Thirteenth EuroSys Conference* (Porto, Portugal) *(EuroSys '18)*. Association for Computing Machinery, Article 32, 12 pages.

[5] Jeremy Chen, Yuqing Huang, Mushi Wang, Semih Salihoglu, and Ken Salem. 2022. Accurate Summary-Based Cardinality Estimation through the Lens of Cardinality Estimation Graphs. *Proceedings of the VLDB Endowment* 15, 8 (April 2022), 1533–1545.

[6] Yannis Chronis, Yawen Wang, Yu Gan, Sami Abu-El-Haija, Chelsea Lin, Carsten Binnig, and Fatma Özcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. arXiv:2408.16170 [cs.DB]

[7] Graham Cormode and S Muthukrishnan. 2004. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. (2004).

[8] Linked Data Benchmark Council. 2024. The LDBC Social Network Benchmark (version 2.2.3). https://arxiv.org/pdf/2001.02299.pdf

[9] Kyle Deeds, Diandre Sabale, Moe Kayali, and Dan Suciu. 2024. Color: A Framework for Applying Graph Coloring to Subgraph Cardinality Estimation. *Proceedings of the VLDB Endowment* 18, 2 (Oct. 2024), 130–143. https://doi.org/10.14778/3705829.3705834

[10] Kyle B. Deeds, Dan Suciu, and Magdalena Balazinska. 2023. SafeBound: A Practical System for Generating Cardinality Bounds. *Proceedings of the ACM on Management of Data* 1, 1 (May 2023), 1–26.

[11] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Fred Zemke. 2022. Graph Pattern Matching in GQL and SQL/PGQ. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2246–2258. https://doi.org/10.1145/3514221.3526057

[12] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. 2007. HyperLogLog: The Analysis of a near-Optimal Cardinality Estimation Algorithm. , 3545 pages.

[13] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 1433–1445. https://doi.org/10.1145/3183713.3190657

[14] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation *(VLDB '21)*, Vol. 15. 752–765.

[15] Hazar Harmouch and Felix Naumann. 2017. Cardinality Estimation: An Experimental Survey. *Proceedings of the VLDB Endowment* 11, 4 (Dec. 2017), 499–512.

[16] Stephen Harris and Nigel Shadbolt. 2005. SPARQL Query Processing with Conventional Relational Database Systems. In *Web Information Systems Engineering – WISE 2005 Workshops*, Mike Dean, Yuanbo Guo, Woochun Jun, Roland Kaschek, Shonali Krishnaswamy, Zhengxiang Pan, and Quan Z. Sheng (Eds.). Springer, Berlin, Heidelberg, 235–244. https://doi.org/10.1007/11581116_25

[17] Shohedul Hasan, Saravanan Thirumuruganathan, Jees Augustine, Nick Koudas, and Gautam Das. 2020. Deep Learning Models for Selectivity Estimation of Multi-Attribute Queries. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 1035–1050.

[18] Axel Hertzschuch, Claudio Hartmann, Dirk Habich, and Wolfgang Lehner. 2021. Simplicity Done Right for Join Ordering. In *11th Biennial Conference on Innovative Data Systems Research (CIDR '21)*.

[19] Pan Hu and Boris Motik. 2024. Accurate Sampling-Based Cardinality Estimation for Complex Graph Queries. *ACM Transactions on Database Systems* 49, 3 (Sept. 2024), 1–46. https://doi.org/10.1145/3689209

[20] Yannis E. Ioannidis and Stavros Christodoulakis. 1993. Optimal Histograms for Limiting Worst-Case Error Propagation in the Size of Join Results. *ACM Transactions on Database Systems* 18, 4 (Dec. 1993), 709–748.

[21] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities:Estimating Correlated Joins with Deep Learning. In *9th Biennial Conference on Innovative Data Systems Research (CIDR '19)*.

[22] Clyde P Kruskal, Larry Rudolph, and Marc Snir. 1990. A complexity theory of efficient parallel algorithms. *Theoretical Computer Science* 71, 1 (1990), 95–132.

[23] Longbin Lai, Yufan Yang, Zhibin Wang, Yuxuan Liu, Haotian Ma, Sijie Shen, Bingqing Lyu, Xiaoli Zhou, Wenyuan Yu, Zhengping Qian, Chen Tian, Sheng Zhong, Yeh-Ching Chung, and Jingren Zhou. 2023. GLogS: Interactive Graph Pattern Matching Query At Large Scale. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 53–69.

[24] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment* 9, 3 (Nov. 2015), 204–215.

[25] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2018. Query Optimization through the Looking Glass, and What We Found Running the Join Order Benchmark. *The VLDB Journal* 27, 5 (Oct. 2018), 643–668.

[26] Feifei Li, Bin Wu, Ke Yi, and Zhuoyue Zhao. 2016. Wander Join: Online Aggregation via Random Walks. In *Proceedings of the 2016 International Conference on Management of Data*. ACM, San Francisco California USA, 615–629.

[27] Yao Lu, Srikanth Kandula, Arnd Christian König, and Surajit Chaudhuri. 2021. Pre-Training Summarization Models of Structured Datasets for Cardinality Estimation. *Proceedings of the VLDB Endowment* 15, 3 (Nov. 2021), 414–426.

[28] Bingqing Lyu, Xiaoli Zhou, Longbin Lai, Yufan Yang, Yunkai Lou, Wenyuan Yu, and Jingren Zhou. 2025. A Modular Graph-Native Query Optimization Framework. In *Proceedings of the 2025 International Conference on Management of Data (SIGMOD '25)*. ACM.

[29] Amine Mhedhbi, Matteo Lissandrini, Laurens Kuiper, Jack Waudby, and Gábor Szárnyas. 2021. LSQB: A Large-Scale Subgraph Query Benchmark. In *Proceedings of the 4th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*. ACM, Virtual Event China, 1–11.

[30] Guido Moerkotte, Thomas Neumann, and Gabriele Steidl. 2009. Preventing Bad Plans by Bounding the Impact of Cardinality Estimation Errors. *Proceedings of the VLDB Endowment* 2, 1 (Aug. 2009), 982–993.

[31] Parimarjan Negi, Ziniu Wu, Andreas Kipf, Nesime Tatbul, Ryan Marcus, Sam Madden, Tim Kraska, and Mohammad Alizadeh. 2023. Robust Query Driven Cardinality Estimation under Changing Workloads. *Proceedings of the VLDB Endowment* 16, 6 (Feb. 2023), 1520–1533.

[32] Thomas Neumann and Guido Moerkotte. 2011. Characteristic Sets: Accurate Cardinality Estimation for RDF Queries with Multiple Joins. In *2011 IEEE 27th International Conference on Data Engineering*. IEEE, Hannover, Germany, 984–994.

[33] Yeonsu Park, Seongyun Ko, Sourav S. Bhowmick, Kyoungmin Kim, Kijae Hong, and Wook-Shin Han. 2020. G-CARE: A Framework for Performance Benchmarking of Cardinality Estimation Techniques for Subgraph Matching. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 1099–1114.

[34] Seyed-Vahid Sanei-Mehri, Ahmet Erdem Sariyuce, and Srikanta Tirthapura. 2018. Butterfly Counting in Bipartite Networks. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, London United Kingdom, 2150–2159.

[35] Tim Schwabe and Maribel Acosta. 2024. Cardinality Estimation over Knowledge Graphs with Embeddings and Graph Neural Networks. *Proceedings of the ACM on Management of Data* 2, 1 (March 2024), 1–26. https://doi.org/10.1145/3639299

[36] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data (SIGMOD '79)*. 23–34.

[37] Haichuan Shang, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2008. Taming Verification Hardness: An Efficient Algorithm for Testing Subgraph Isomorphism. *Proceedings of the VLDB Endowment* 1, 1 (Aug. 2008), 364–375. https://doi.org/10.14778/1453856.1453899

[38] Giorgio Stefanoni, Boris Motik, and Egor V. Kostylev. 2018. Estimating the Cardinality of Conjunctive Queries over RDF Data Using Graph Summarisation. In *Proceedings of the 2018 World Wide Web Conference* (Lyon, France) *(WWW '18)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 1043–1052.

[39] Giorgio Stefanoni, Boris Motik, and Egor V. Kostylev. 2018. Estimating the Cardinality of Conjunctive Queries over RDF Data Using Graph Summarisation. In *Proceedings of the 2018 World Wide Web Conference on World Wide Web - WWW '18 (WWW '18)*. 1043–1052. https://doi.org/10.1145/3178876.3186003 arXiv:1801.09619 [cs]

[40] Ji Sun, Jintao Zhang, Zhaoyan Sun, Guoliang Li, and Nan Tang. 2021. Learned Cardinality Estimation: A Design Space Exploration and a Comparative Evaluation. *Proceedings of the VLDB Endowment* 15, 1 (Sept. 2021), 85–97.

[41] Charalampos E. Tsourakakis, Petros Drineas, Eirinaios Michelakis, Ioannis Koutis, and Christos Faloutsos. 2011. Spectral Counting of Triangles via Element-Wise Sparsification and Triangle-Based Link Recommendation. *Social Network Analysis and Mining* 1, 2 (April 2011), 75–81.

[42] Wilco Van Leeuwen, George Fletcher, and Nikolay Yakovets. 2022. A General Cardinality Estimation Framework for Subgraph Matching in Property Graphs.

IEEE Transactions on Knowledge and Data Engineering (2022), 1–1.

[43] David Vengerov, Andre Cavalheiro Menck, Mohamed Zait, and Sunil P. Chakkappen. 2015. Join Size Estimation Subject to Filter Conditions. *Proceedings of the VLDB Endowment* 8, 12 (Aug. 2015), 1530–1541.

[44] Jiayi Wang, Chengliang Chai, Jiabin Liu, and Guoliang Li. 2021. FACE: A Normalizing Flow Based Cardinality Estimator. *Proceedings of the VLDB Endowment* 15, 1 (Sept. 2021), 72–84.

[45] Xiaoying Wang, Changbo Qu, Weiyuan Wu, Jiannan Wang, and Qingqing Zhou. 2021. Are We Ready for Learned Cardinality Estimation? *Proceedings of the VLDB Endowment* 14, 9 (May 2021), 1640–1654.

[46] Zhengdong Wang, Qiang Yin, and Longbin Lai. 2024. *Path-centric Cardinality Estimation for SubGraph Matching.* https://cs.sjtu.edu.cn/~qyin/pathce.pdf.

[47] Leonard Wörteler, Moritz Renftle, Theodoros Chondrogiannis, and Michael Grossniklaus. 2022. Cardinality Estimation Using Label Probability Propagation for Subgraph Matching in Property Graph Databases *(EDBT '22).* OpenProceedings.org. https://doi.org/10.48786/EDBT.2022.16

[48] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1, 1 (May 2023), 1–27.

[49] Ziniu Wu, Amir Shaikhha, Rong Zhu, Kai Zeng, Yuxing Han, and Jingren Zhou. 2021. BayesCard: Revitilizing Bayesian Frameworks for Cardinality Estimation. arXiv:2012.14743 [cs.DB]

[50] Tianjing Zeng, Junwei Lan, Jiahong Ma, Wenqing Wei, Rong Zhu, Pengfei Li, Bolin Ding, Defu Lian, Zhewei Wei, and Jingren Zhou. 2024. PRICE: A Pretrained Model for Cross-Database Cardinality Estimation. arXiv:2406.01027 [cs.DB]

[51] Kangfei Zhao, Jeffrey Xu Yu, Hao Zhang, Qiyan Li, and Yu Rong. 2021. A Learned Sketch for Subgraph Counting. In *Proceedings of the 2021 International Conference on Management of Data.* ACM, Virtual Event China, 2142–2155. https://doi.org/10.1145/3448016.3457289

[52] Zhuoyue Zhao, Robert Christensen, Feifei Li, Xiao Hu, and Ke Yi. 2018. Random Sampling over Joins Revisited. In *Proceedings of the 2018 International Conference on Management of Data.* ACM, Houston TX USA, 1525–1539.

[53] Rong Zhu, Ziniu Wu, Yuxing Han, Kai Zeng, Andreas Pfadler, Zhengping Qian, Jingren Zhou, and Bin Cui. 2021. FLAT: Fast, Lightweight and Accurate Method for Cardinality Estimation. *Proc. VLDB* 14, 9 (May 2021), 1489–1502.

A A COST-BASED PRUNING STRATEGY

If GenEstOrder fails for a PSG query $\mathcal{Q}$ of Q, we prune edges from Q to obtain a subquery Q', such that its PSG query $\mathcal{Q}'$ has a feasible estimation order $\mathcal{S}'$. Estimation is then performed using $(\mathcal{Q}', \mathcal{S}')$. To ensure the result is still an upper bound on $|Q(G)|$, only edges within a cycle of Q are pruned. We next outline the *cost-based* edge pruning strategy of PathCE.

(1) We first define a cost metric to quantify the impact of pruning an edge. Let $e = (u_1, u_2)$ be an edge in Q. Denote by APF_e the the *amplification factor* of pruning e from Q, which is defined as

$$\mathsf{APF}_e = \mathsf{Card}(u_1) \cdot \mathsf{Card}(u_2)/\mathsf{Card}(e) \tag{6}$$

where $\mathsf{Card}(u_1)$, $\mathsf{Card}(u_2)$, and $\mathsf{Card}(e)$ are the match counts for u_1, u_2, and e in G, respectively.

(2) Based on this metric, PathCE prunes as follows. Let u be the node where GenEstOrder fails, and $u_1, \ldots, u_t$ be its neighbors in $\mathcal{Q}$, with $t \geq 3$. Let $P_1, \ldots, P_t$ be the paths that connect u to $u_1, \ldots, u_t$ in $\mathcal{Q}$. Each P_i should be part of some cycle in Q. PathCE selects $t-2$ edges, $e_{i_1}, \ldots, e_{i_{t-2}}$, to prune such that (i) $\prod_{k=1}^{t-2} \mathsf{APF}_{e_{i_k}}$ is minimized, and (ii) each e_{i_k} comes from a distinct path P_{i_k}. Intuitively, this is to minimize the impact while breaking $t-2$ paths linked to u.

Afterward, PathCE recomputes $\mathcal{Q}$ and restarts GenEstOrder (see Algorithm 2, line 4). The disconnected paths will be processed before u by GenEstOrder, ensuring that it doesn't fail on u again.

Example 7: Assume that GenEstOrder fails on node u_1 in PSG query $\mathcal{Q}$ depicted in Fig. 5(a). There are three paths in Q, P_1, P_2, and P_3, connecting u_1 to its neighbors u_3, u_6, and u_5, respectively. Then PathCE disconnects one path based on the amplification factors of edges on these paths, *i.e.,* (u_1, u_2), (u_2, u_3), (u_1, u_6) and (u_1, u_5). Assuming (u_1, u_2) has the minimal amplification factor, PathCE prunes (u_1, u_2) from Q to break P_1. The revised query Q' and PSG query $\mathcal{Q}'$ are shown in Fig. 5(b), from which a feasible estimation order $\mathcal{S}' = [u_2, u_3, u_6, u_5]$ can be obtained by GenEstOrder. □

B SUPPLEMENTARY MATERIAL FOR EXP-1

In this section, we present the true cardinalities as well as the estimated cardinalities produced by all baselines on LDBC, IMDB, and AIDS. We also include results from two additional baselines: PathCE$^-$ and ColorMax. Here, PathCE$^-$ denotes a variant of PathCE that uses at most 2-path query statistics (*i.e.,* $K = 2$), and ColorMax refers to the MaxMix32 variant of Color. Note that unlike Color, ColorMax is a pessimistic estimator and always produces upper bound estimates.

(1) The results on all datasets are presented in Tables 3, 4, and 5. More specifically:

- The results on LDBC are presented in Table 3.
- The results on IMDB are presented in Table 4.
- The results on AIDS are presented in Table 5.

In Tables 3, 4 and 4, we set the estimated cardinality as 1 if the estimator exceeds the time limit (5 minutes) or encounters a sampling failure. Specifically, we mark the cardinality in timeout case with $*$ and the cardinality in sampling failure case with †.

(2) To further demonstrate the advantage of PathCE, we also provide bar plots comparing PathCE with PathCE$^-$, FactorJoin, and ColorMax, as shown in Fig. 8.

Table 3: Cardinalities on LDBC queries.

(a) LDBC Cyclic

Query	Q2	Q3	Q8	Q9	Q10	Q11
TrueCard	1.02×10^6	7.07×10^5	3.46×10^6	7.08×10^7	4.42×10^4	2.22×10^4
PathCE	1.19×10^6	1.33×10^7	6.29×10^6	1.66×10^9	1.09×10^6	2.63×10^5
PathCE$^-$	1.19×10^6	1.58×10^7	1.66×10^7	2.72×10^9	2.9×10^6	9.85×10^5
GLogS	7.2×10^3	28.6	1.9×10^5	2.08×10^7	4.42×10^4	1.23×10^3
FactorJoin	1.19×10^6	2.18×10^9	4.15×10^7	7.15×10^9	2.9×10^6	1.62×10^6
CEG	7.87×10^7	7.78×10^5	3.34×10^6	7.07×10^7	4.42×10^4	1.62×10^6
SumRDF	5.58×10^3	98.8	1.72×10^3	2.27×10^6	6.3×10^3	1.37×10^3
WJ	7.89×10^5	3.28×10^5	3.23×10^6	6.83×10^7	2.28×10^4	2×10^4
GNCE	9.19×10^3	1.22	87.2	1.29×10^6	2.66	189
Color	2.28×10^6	1	72.6	4.13×10^7	2.42×10^4	1
ColorMax	3.54×10^{10}	3.29×10^{10}	5.39×10^9	7.38×10^{10}	7.02×10^7	2.52×10^9
Query	Q14	Q15	Q16	Q17	Q19	Q20
TrueCard	200	2.98×10^6	9.9×10^6	6.24×10^4	6.28×10^5	2.67×10^7
PathCE	1.64×10^4	1.28×10^8	2.23×10^9	2.8×10^6	2.16×10^8	3.25×10^{11}
PathCE$^-$	7.93×10^5	2.11×10^8	2.61×10^9	2.8×10^6	4.56×10^{10}	6.41×10^{14}
GLogS	1	1	2.17×10^4	278	935	661
FactorJoin	1.2×10^6	2.2×10^8	1.19×10^{11}	1.4×10^7	4.3×10^{12}	3.86×10^{18}
CEG	180	3.17×10^6	6.34×10^9	3.64×10^7	2.21×10^9	1^*
SumRDF	1	14.7	6.76×10^3	2.22	1^*	1^*
WJ	1.71	$1^\dagger$	$1^\dagger$	$1^\dagger$	6.76×10^4	$1^\dagger$
GNCE	4.36	4.44	14	10.4	1.85×10^3	9.93×10^4
Color	83.3	8.78×10^3	6.42×10^6	1	1	1
ColorMax	1.63×10^8	2.92×10^{12}	9.69×10^{15}	3.81×10^{11}	3.96×10^{14}	2.04×10^{20}

(b) LDBC Acyclic.

Query	Q1	Q4	Q5	Q6	Q13
TrueCard	2.17×10^8	5.5×10^6	6.29×10^6	1.66×10^9	1.81×10^8
PathCE	6.98×10^8	5.5×10^6	6.29×10^6	1.66×10^9	5.65×10^8
PathCE$^-$	1.57×10^9	5.5×10^6	1.66×10^7	4.83×10^9	7.37×10^8
GLogS	3.5×10^7	1.47×10^6	2.13×10^6	1.04×10^9	4.38×10^7
FactorJoin	5.6×10^{13}	1.17×10^8	1.35×10^8	8.67×10^9	6.39×10^9
CEG	7.59×10^7	5.77×10^6	4.33×10^6	1.65×10^9	6.55×10^7
SumRDF	4.67×10^7	5.38×10^6	4.33×10^6	4.63×10^8	4.03×10^7
WJ	2.11×10^8	5.72×10^6	6.31×10^6	1.68×10^9	1.8×10^8
GNCE	1.68×10^5	8.02×10^7	4.04×10^4	1.46×10^8	2.68×10^5
Color	4.3	3.96×10^5	4.29×10^5	6.07×10^8	5.39×10^6
ColorMax	3.89×10^{12}	3.54×10^{10}	5.39×10^9	7.38×10^{10}	6.68×10^{11}

Table 4: Cardinalities on IMDB queries.

(a) Query with size ≤ 4

Query	1	2	3	4	5	6	8
TrueCard	4.07×10^6	3.49×10^7	2.35×10^8	1.04×10^7	6.74×10^7	2.16×10^8	8.72×10^7
PathCE	4.07×10^6	3.49×10^7	2.35×10^8	1.04×10^7	6.74×10^7	2.16×10^8	1.47×10^9
PathCE$^-$	4.07×10^6	3.49×10^7	2.35×10^8	1.04×10^7	6.74×10^7	2.21×10^9	2×10^9
GLogS	4.07×10^6	3.49×10^7	2.35×10^8	1.04×10^7	6.74×10^7	1.15×10^8	1.95×10^7
FactorJoin	9.48×10^6	2.53×10^8	2.78×10^9	1.71×10^7	7.68×10^8	5.02×10^9	2.75×10^{11}
CEG	4.07×10^6	3.49×10^7	2.35×10^8	1.04×10^7	6.74×10^7	2.16×10^8	8.07×10^7
SumRDF	3.69×10^6	2.01×10^7	1.4×10^8	1.04×10^7	3.99×10^7	1.64×10^8	4.51×10^7
WJ	3.65×10^6	2.99×10^7	2.13×10^8	1.01×10^7	5.68×10^7	2.15×10^8	7.81×10^7
GNCE	1×10^5	1.19×10^5	6.71×10^5	5.12×10^4	1.34×10^6	8.28×10^5	5.75×10^5
Color	1.42×10^5	2.14×10^5	4.93×10^6	3.47×10^5	4.12×10^5	6.47×10^6	1.94×10^5
ColorMax	6.22×10^8	6.24×10^9	1.1×10^{10}	4.33×10^8	8.37×10^9	5.4×10^{10}	9.08×10^{12}
Query	10	11	12	13	14	15	17
TrueCard	3.66×10^7	4.14×10^6	1.77×10^8	1.77×10^8	6.96×10^8	1.7×10^{10}	3.1×10^9
PathCE	3.66×10^7	4.14×10^6	1.77×10^8	1.77×10^8	6.96×10^8	1.7×10^{10}	1.77×10^{10}
PathCE$^-$	4.1×10^7	4.14×10^6	1.77×10^8	1.77×10^8	6.96×10^8	1.7×10^{10}	1.77×10^{10}
GLogS	2.51×10^7	3.39×10^5	3.03×10^7	3.03×10^7	8.29×10^7	2.7×10^7	1.69×10^8
FactorJoin	8.27×10^8	1.43×10^8	2.9×10^9	2.9×10^9	1.08×10^{10}	1.96×10^{12}	2.39×10^{11}
CEG	3.89×10^7	2.22×10^6	1.05×10^8	1.05×10^8	5.42×10^8	2.63×10^9	1.66×10^9
SumRDF	2.81×10^7	1.12×10^6	9.79×10^7	9.79×10^7	4×10^8	4.03×10^9	1.1×10^9
WJ	3.31×10^7	2.55×10^6	1.46×10^8	1.46×10^8	6.65×10^8	1.33×10^{10}	2.66×10^9
GNCE	2.72×10^5	137	204	224	104	1.05	2.58×10^3
Color	3.72×10^6	4.09×10^5	1.15×10^6	1.15×10^6	3.95×10^6	8.65×10^5	5.19×10^7
ColorMax	2.23×10^{10}	3.1×10^{12}	2.84×10^{11}	2.84×10^{11}	1.21×10^{12}	7.23×10^{13}	2.17×10^{13}
Query	18	21	22	23	32		
TrueCard	1.12×10^9	8.82×10^8	1.51×10^{10}	5.53×10^9	2.88×10^5		
PathCE	1.01×10^{10}	8.82×10^8	1.51×10^{10}	5.53×10^9	2.88×10^5		
PathCE$^-$	1.01×10^{10}	8.82×10^8	1.51×10^{10}	5.53×10^9	2.88×10^5		
GLogS	1.93×10^8	1.8×10^6	7.34×10^7	7.93×10^6	2.88×10^5		
FactorJoin	4.21×10^{10}	4.08×10^{10}	6.38×10^{11}	3.25×10^{11}	4.21×10^6		
CEG	6.48×10^8	1.16×10^8	4.18×10^9	8.71×10^8	2.88×10^5		
SumRDF	8.14×10^8	5.02×10^7	3.81×10^9	1.64×10^9	2.02×10^5		
WJ	1.09×10^9	4.7×10^8	1.12×10^{10}	4.12×10^9	2.78×10^5		
GNCE	1.5×10^3	1.02	1.08	1.48	1.02×10^5		
Color	4.24×10^7	3.47×10^6	2.89×10^7	6.14×10^6	1.88×10^4		
ColorMax	5.06×10^{12}	1.61×10^{15}	6.82×10^{13}	8.28×10^{12}	5.67×10^9		

(b) Query with size > 4

Query	7	9	16	19	20	24	25
TrueCard	7.98×10^7	4.66×10^7	3.19×10^9	3.13×10^9	8.23×10^7	3.37×10^{11}	7.88×10^{10}
PathCE	1.6×10^9	1.15×10^9	2.68×10^{10}	7.89×10^{10}	8.48×10^7	8.67×10^{12}	3.52×10^{11}
PathCE$^-$	5.99×10^{10}	1.15×10^9	7.66×10^{11}	7.89×10^{10}	8.48×10^7	8.67×10^{12}	3.52×10^{11}
GLogS	2.67×10^5	7.76×10^6	4.35×10^7	4.52×10^7	3.14×10^6	9.74×10^7	5.47×10^8
FactorJoin	2.27×10^{12}	4.91×10^{10}	4.44×10^{13}	2.23×10^{13}	5.1×10^9	5.88×10^{15}	1.03×10^{13}
CEG	2.12×10^7	3.91×10^7	1.67×10^9	1.02×10^9	5.02×10^7	4.22×10^{10}	2.59×10^{10}
SumRDF	7.86×10^6	2.55×10^7	8.43×10^8	8.92×10^8	6.81×10^7	4.75×10^{10}	2.62×10^{10}
WJ	8.06×10^7	4.15×10^7	2.7×10^9	2.56×10^9	8.18×10^7	2.41×10^{11}	7.46×10^{10}
GNCE	1.35	1.15	1.11×10^3	1.27	1.02	1.82	1.35
Color	4.26×10^4	6.98×10^4	1.19×10^6	7.02×10^5	1.23×10^6	9.59×10^6	3.13×10^8
ColorMax	1.15×10^{15}	1.09×10^{14}	3.88×10^{15}	7.14×10^{16}	1.21×10^{12}	3.96×10^{19}	2.67×10^{15}
Query	26	27	28	29	30	31	33
TrueCard	2.48×10^8	1.67×10^8	1.76×10^{10}	1.75×10^{13}	8.58×10^{10}	2.13×10^{12}	7.9×10^6
PathCE	2.56×10^8	1.67×10^8	1.76×10^{10}	1.14×10^{15}	3.64×10^{11}	7.7×10^{12}	5.81×10^7
PathCE$^-$	2.56×10^8	1.67×10^8	1.76×10^{10}	1.14×10^{15}	3.64×10^{11}	7.7×10^{12}	5.81×10^7
GLogS	1.76×10^6	9.14×10^4	3.82×10^6	5.35×10^6	3.41×10^7	6.14×10^8	3.65×10^4
FactorJoin	1.95×10^{10}	8.14×10^{10}	1.28×10^{12}	4.2×10^{18}	2.06×10^{13}	6.15×10^{14}	5.14×10^8
CEG	1.16×10^8	5.55×10^7	2.01×10^9	2.31×10^{11}	1.24×10^{10}	1.99×10^{11}	9.4×10^6
SumRDF	2.04×10^8	2.99×10^7	4.98×10^9	4.73×10^{11}	3.3×10^{10}	3.04×10^{11}	4.26×10^6
WJ	2.5×10^8	1.76×10^8	1.33×10^{10}	1.18×10^{13}	8.09×10^{10}	1.66×10^{12}	4.48×10^6
GNCE	1.66	1.4	1.39	2.22	1.22	1.29	5.16
Color	1.2×10^6	7.48×10^5	5.39×10^6	2.46×10^7	5.99×10^7	9.37×10^8	2.1×10^3
ColorMax	1.88×10^{13}	3.21×10^{15}	1.36×10^{14}	1.3×10^{22}	5.33×10^{15}	2.84×10^{17}	2.89×10^{13}

Table 5: Statistics of q-error on AIDS queries.

(a) Path

Baseline	Min	Max	Mean	P25	P50 (Median)	P75
PathCE	1	8.35×10^3	94.2	1	3.44	9.92
PathCE$^-$	1	7.51×10^4	909	7.46	17.7	57.1
GLogS	1	1.09×10^3	44.4	1.97	5.4	17.3
FactorJoin	2.64	1.1×10^8	9.58×10^5	73.4	858	5.3×10^3
CEG	1	32.7	2.33	1.09	1.6	2.19
SumRDF	1	1.18×10^8	1.2×10^7	1.33	1.75	3.03
WJ	1	4.58×10^4	289	1.03	1.05	1.25
GNCE	1.01	1.16×10^5	735	2.48	4.26	15.8
Color	1	327	9.98	1.64	4.38	10
ColorMax	213	5.28×10^{12}	3.98×10^{10}	1.91×10^4	2.5×10^6	1.05×10^8

(b) Star

Baseline	Min	Max	Mean	P25	P50 (Median)	P75
PathCE	1	3×10^5	6.52×10^3	1	30.2	60.7
PathCE$^-$	1	3×10^5	6.52×10^3	1	30.2	60.7
GLogS	1	4.41×10^3	47.7	1	2.21	23
FactorJoin	2.64	3×10^5	1.23×10^4	2.64	30.2	227
CEG	1	3.57×10^9	3×10^7	1	1.11	3.03
SumRDF	1	3.57×10^9	3×10^7	1.16	1.34	2.4
WJ	1	114	2.91	1.01	1.02	1.03
GNCE	1.19	1.01×10^7	2.46×10^5	3.69	10.8	112
Color	1.01	6.44	2.17	1.54	1.54	1.77
ColorMax	213	9.54×10^{11}	3.21×10^{10}	213	1.36×10^5	2.99×10^6

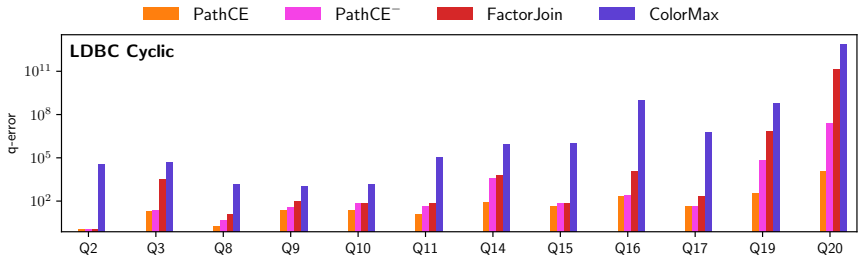
(c) Tree

Baseline	Min	Max	Mean	P25	P50 (Median)	P75
PathCE	1	1.95×10^5	1.79×10^3	1	11.3	63.5
PathCE$^-$	1	1.95×10^5	2.25×10^3	4.31	18	129
GLogS	1	1.66×10^3	99.2	1.17	8.91	41.2
FactorJoin	2.64	1.08×10^7	1.22×10^5	22.3	170	1.95×10^3
CEG	1	57.8	3.08	1.07	1.36	2.66
SumRDF	1	6.7×10^8	4.04×10^7	1.34	1.56	4.82
WJ	1	224	3.02	1.01	1.05	1.19
GNCE	1.01	3.22×10^5	2.5×10^3	3.01	8.93	31.6
Color	1.01	80.5	5.01	1.54	2.46	5.24
ColorMax	213	8.32×10^{11}	8.73×10^9	9.58×10^3	5.37×10^5	7.53×10^7

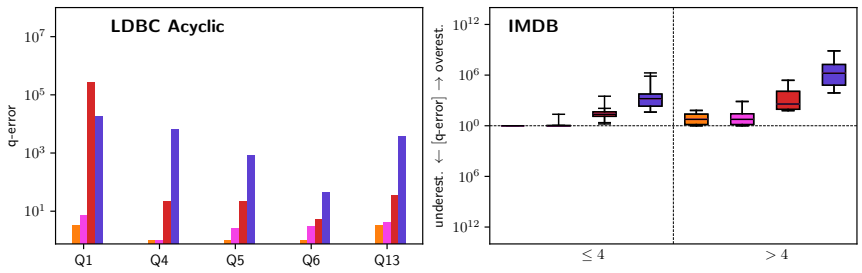
(d) Graph

Baseline	Min	Max	Mean	P25	P50 (Median)	P75
PathCE	1	404	55.1	1	1	34.2
PathCE$^-$	1	479	74.7	1	1.45	167
GLogS	1	1.38×10^6	2.67×10^4	1	1	5.5
FactorJoin	2.64	8.6×10^3	822	2.64	17.5	459
CEG	1	199	5.44	1	1	1.3
SumRDF	1	8.7×10^7	3×10^6	1.34	1.34	230
WJ	1	1.39×10^4	230	1.01	1.01	1.17
GNCE	1.03	2.75×10^6	4.96×10^4	1.51	3.73	124
Color	1.01	39.1	3.29	1.54	1.54	2.25
ColorMax	213	3.51×10^8	1.08×10^7	213	1.4×10^3	1.99×10^5

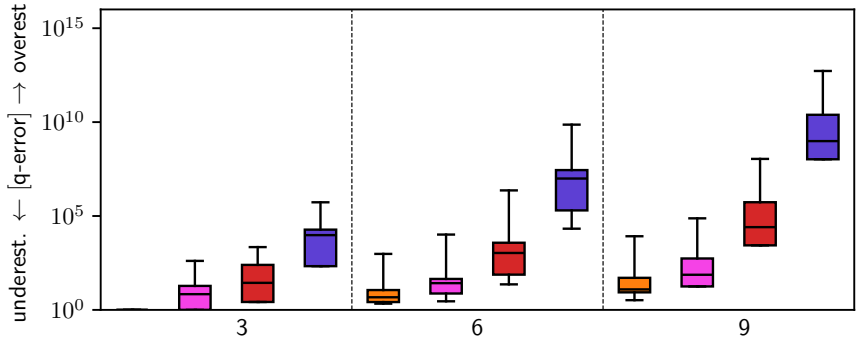
Here we also provide bar plots to illustrate the superiority of PathCE over other pessimistic estimators, as shown in Fig. 8.



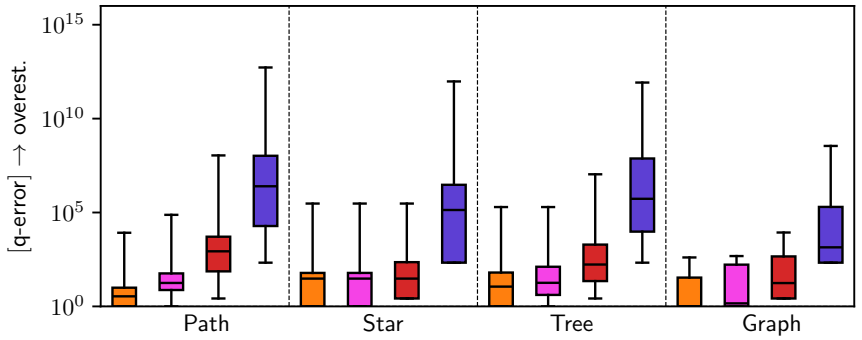
(a) Accuracy (varying query, LDBC cyclic)



(b) Accuracy (varying query, LDBC acyclic and IMDB)



(c) Accuracy (varying path query size, AIDS)



(d) Accuracy (varying query topology, AIDS)

Figure 8: Estimation accuracy evaluation of pessimistic estimators.

C SUPPLEMENTARY MATERIAL FOR EXP-5

In this section, we present detailed planning and execution time in the end-to-end performance evaluation, as shown in Table 6. **P** and **E** represent aggregated planning and execution time on LDBC/IMDB, respectively, and **P + E** is the end-to-end time. The time is measured in seconds.

Table 6: End-to-end performance.

Baseline	LDBC			IMDB		
	P	**E**	**P + E**	**P**	**E**	**P + E**
PathCE	3.89	79.92	83.81	1.54	4080.15	4081.68
GLogS	319.64	839.52	1159.17	0.50	3635.4	3635.89
FactorJoin	197.83	115.19	313.02	3.29	4264.04	4267.32
CEG	3292.14	904.80	4196.94	109.47	4079.41	4188.87
SumRDF	24428.7	3792.02	28220.8	1399.45	3614.54	5013.99
WJ	29.54	176.23	205.77	619.24	4080.14	4699.39
Color	0.44	3845.35	3845.79	0.36	4094.39	4094.75
ColorMax	0.49	2149.1	2149.59	0.29	7010.13	7010.42

From the results, we find the following.

(1) PathCE beats all other baselines by 2.46× ∼ 336.71× on LDBC in end-to-end time. PathCE is able to generate the best query plans especially for complex queries like Q19 and Q20 thanks to its accurate upper bound estimates. Thus, PathCE beats all baselines by 1.44× ∼ 48.11× in execution time. PathCE also requires less planning time than all baselines except Color and ColorMax, which, while fastest in planning, suffer from suboptimal plans. Compared to FactorJoin, PathCE achieves a 3.73× speedup in E2E time, with 50.85× and 1.44× improvements in planning and query execution, respectively.

(2) On IMDB, all baselines except ColorMax show similar execution performance, as the queries are acyclic and simple. PathCE achieves 2.14× ∼ 911.63× lower planning latency than FactorJoin, CEG, SumRDF, and WJ, and remains comparable to GLogS, Color and ColorMax (gap < 1.5s, negligible relative to end-to-end time).

(3) On LDBC, since ColorMax's estimates are guaranteed to be cardinality upper bounds, it is able to avoid bad plans for complex queries like Q19 (see next supplementary material for more details) and Q20, and thus outperforming Color, which times out for such queries.

D SUPPLEMENTARY MATERIAL FOR END-TO-END PERFORMANCE ON LDBC Q19

In this section, we present a case study on the end-to-end performance of all baselines on LDBC query $Q19$, a query with two diamond subqueries. The detailed planning and execution time are shown in Table 7. And the generated plans from each estimator are illustrated in Fig. 9.

Table 7: End-to-end performance on LDBC query $Q19$.

Baseline	P	E	P + E
PathCE	0.32	0.88	1.20
GLogS	0.10	9.19	9.29
FactorJoin	5.92	2.07	7.99
CEG	5.71	0.88	6.59
SumRDF	2526.94	29.82	2556.76
WJ	2.51	0.88	3.39
Color	0.03	29.06	29.08
ColorMax	0.02	2.07	2.09

Regarding planning, PathCE performs only slightly slower than GLogS, Color and ColorMax, but much faster than other baselines. For query execution, PathCE, CEG and WJ share the best plan. FactorJoin's and ColorMax's plan differs slightly from PathCE's plan but does not scale as well as PathCE. Unlike other estimators, GLogS, SumRDF and Color produce underestimations for Q19, resulting in suboptimal expansion-only plans with large intermediate results. The upper bound estimation of PathCE, FactorJoin and ColorMax helps prevent this issue. CEG and WJ do not face the issue due to more accurate estimation.

Combining planning and querying, this study clearly showcases PathCE's superiority for optimizing graph query execution.

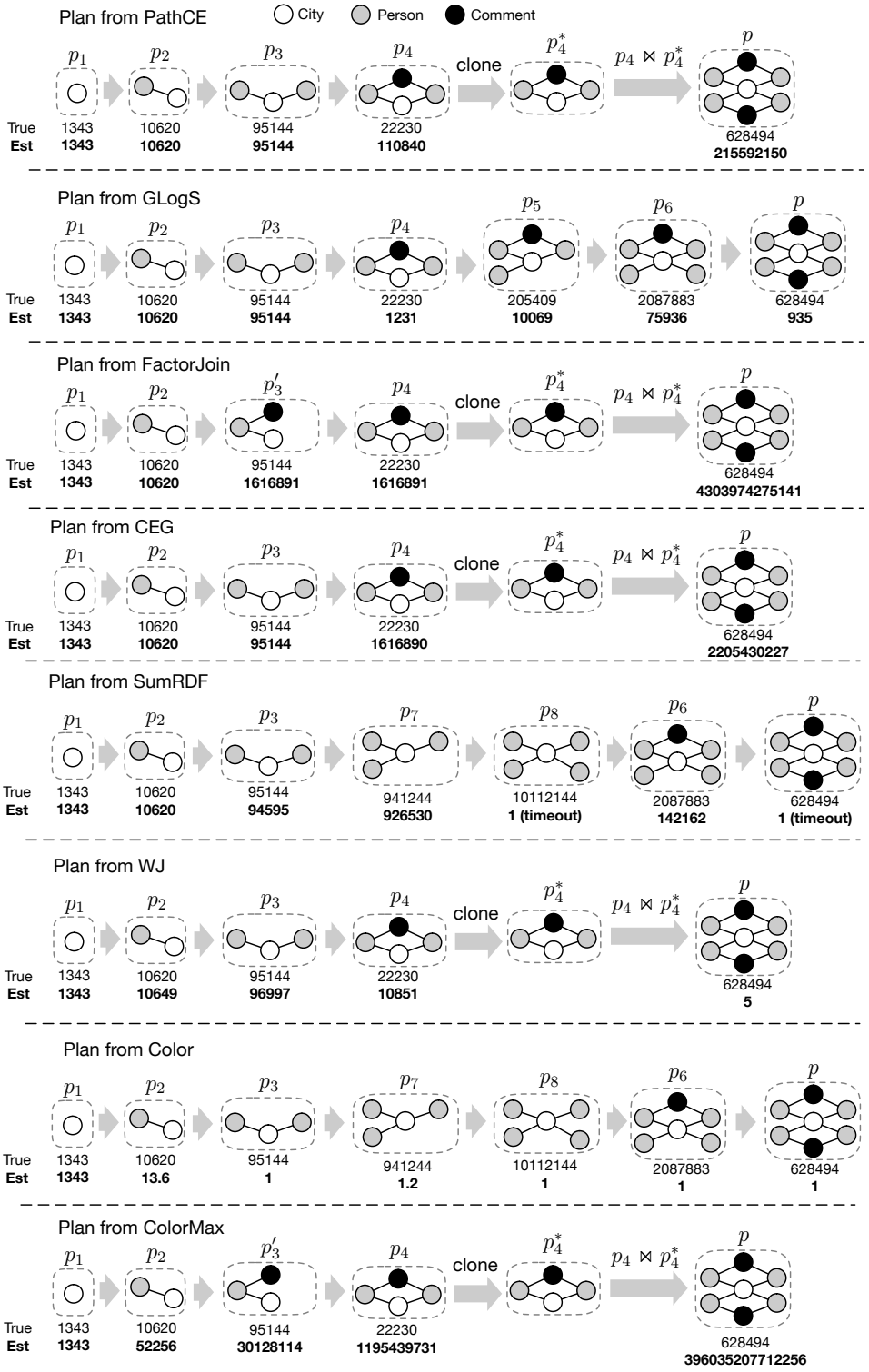


Figure 9: The generated plans from each estimator for LDBC query Q19, where true and estimated cardinalities for a subquery of Q19 are listed below its corresponding subquery graph. It should be noted that Color's plan shares the same set of subqueries with SumRDF's plan, but with a slight difference on the expand order before intersection. Since this has negligible impact on the execution time, we omit such difference and focus on the cardinalities of subqueries.